



Universidad Politécnica
de Madrid

**Escuela Técnica Superior de
Ingenieros Informáticos**



Máster Universitario en Inteligencia Artificial

Trabajo Fin de Máster

**Entorno de Aprendizaje por Refuerzo
sobre Aerostack2 para el Control de
Velocidad de un Dron**

Autor: Jordi Florit Ensenyat
Tutor: Javier Melero Deza

Madrid, Julio - 2026

Este Trabajo Fin de Máster se ha depositado en la ETSI Informáticos de la Universidad Politécnica de Madrid para su defensa.

Trabajo Fin de Máster

Máster: Máster Universitario en Inteligencia Artificial

Título: Entorno de Aprendizaje por Refuerzo sobre Aerostack2 para el Control de Velocidad de un Dron

Julio - 2026

Autor: Jordi Florit Ensenyat

Tutor: Javier Melero Deza

Departamento de Inteligencia Artificial

Escuela Técnica Superior de Ingenieros Informáticos

Universidad Politécnica de Madrid

Índice general

1. Introducción	3
1.1. Motivación del proyecto	3
1.2. Contexto del proyecto	4
1.3. Objetivos	5
1.4. Estructura del documento	5
2. Fundamentos técnicos y trabajo relacionado	6
2.1. Infraestructura robótica: ROS 2 y Aerostack2	6
2.1.1. ROS 2 como middleware robótico	6
2.1.2. Aerostack2	7
2.1.3. Gymnasium y Stable-Baselines3 en el flujo de aprendizaje	9
2.1.3.0.1. Herramientas de observación y monitorización.	9
2.2. Aprendizaje por refuerzo para el control de UAV	10
2.2.1. PPO como algoritmo de entrenamiento	13
2.3. Trabajo relacionado: Simuladores y entornos de entrenamiento para RL en drones	15
2.4. Síntesis y posicionamiento del trabajo	17
3. Diseño del sistema	18
3.1. Arquitectura general	18
3.2. Formulación del problema de control	19
3.3. Diseño del entorno Gymnasium	21
3.4. Diseño del reinicio de episodios y accionabilidad	23
3.5. Diseño de la recompensa y condiciones de finalización	24
3.6. Guards de acción y protección de fronteras	25
3.7. Diseño del curriculum de distancia	26
3.8. Diseño de la ejecución vectorizada	26
3.9. Diseño del protocolo de evaluación	28
4. Implementación	29
4.1. Organización del software	29
4.2. Integración con Aerostack2 y ROS 2	30
4.3. Configuración operativa de PPO	32
4.4. Herramientas de visualización y monitorización	33
4.5. Implementación del entorno de aprendizaje	35
4.6. Diagnóstico del reinicio y de la accionabilidad	36
4.7. Infraestructura de entrenamiento y vectorización	37

5. Pruebas y evaluación	39
5.1. Estrategia de validación	39
5.2. Entorno experimental	40
5.3. Pruebas deterministas y de configuración	40
5.4. Pruebas con Aerostack2 y reinicio de episodios	42
5.5. Métricas y criterios de aceptación	44
6. Resultados	45
6.1. Validación funcional de la infraestructura	45
6.2. Matriz de experimentos PPO	45
6.3. Aprendizaje mediante curriculum de distancia	47
6.4. Evaluación formal y comparación con un controlador PID	48
6.5. Validación del entrenamiento vectorizado	50
6.6. Hallazgos sobre el simulador	51
6.7. Discusión de limitaciones	52
7. Impacto del trabajo	53
7.1. Impacto técnico y académico	53
7.2. Impacto social e industrial	53
8. Conclusiones y líneas futuras	55
8.1. Conclusiones	55
8.2. Líneas futuras	56
Bibliografía	58
A. Anexos	61
A.1. Configuraciones representativas	61
A.2. Comandos representativos	62
A.3. Capturas de TensorBoard	64
A.4. Declaración de uso de herramientas de inteligencia artificial	64

Índice de Figuras

2.1. Comunicación entre nodos en ROS mediante <i>topics</i> y servicios. Fuente: Chollet [Chollet, 2021].	7
2.2. Organización por capas de los componentes software de Aerostack2. Fuente: Fernández-Cortizas et al. [Fernández-Cortizas et al., 2023].	8
2.3. Relación conceptual entre aprendizaje por refuerzo clásico, aprendizaje profundo supervisado y aprendizaje por refuerzo profundo. Fuente: Objectways [Objectways, 2025].	11
2.4. Bucle de interacción entre agente y entorno en aprendizaje por refuerzo. Fuente: AltexSoft [AltexSoft, 2021], basado en la formulación clásica de Sutton y Barto [Sutton y Barto, 2018].	11
2.5. Arquitectura actor-crítico utilizada habitualmente en PPO. Fuente: ResearchGate [ResearchGate, 2024].	14
3.1. Arquitectura general del sistema, separando la lógica de aprendizaje, la interfaz Gymnasium, la comunicación robótica con Aerostack2 y el simulador con servicio de reinicio.	19
3.2. Representación conceptual de la pose de un dron mediante los ángulos de <i>roll</i> , <i>pitch</i> y <i>yaw</i> . Fuente: ResearchGate.	20
3.3. Relación geométrica entre el UAV, el objetivo relativo, la dirección de avance y la orientación de guiñada utilizada en el criterio de <i>path-facing</i>	21
3.4. Ciclo de interacción entre el agente, el entorno Gymnasium y el simulador durante un episodio.	22
3.5. Diseño de la ejecución vectorizada mediante varias instancias lógicas asociadas a espacios de nombres diferenciados sobre una infraestructura de simulación compartida.	27
4.1. Integración implementada entre la política de aprendizaje, la interfaz Gymnasium, Aerostack2 y ROS 2.	31
4.2. Visualización en RViz2 del UAV simulado, la trayectoria reciente y el marcador del objetivo durante una ejecución con Aerostack2.	34
4.3. Ciclo implementado de un episodio de aprendizaje por refuerzo sobre AS2TestEnv.	36
6.1. Tasa de éxito por experimento durante la campaña principal. Cada barra agrega todos los episodios registrados por el monitor vectorizado del experimento; <i>n</i> indica el número de episodios.	47

6.2. Tasa de éxito en ventana móvil de 50 episodios a lo largo de las tres etapas del curriculum y del entrenamiento vectorizado posterior. Las líneas discontinuas marcan los cambios de etapa, en los que se amplía el rango de distancias muestreado.	48
6.3. Tasa de éxito de los cinco controladores evaluados sobre los mismos 100 episodios sembrados.	49
6.4. Distribución de la distancia final al objetivo en la evaluación determinista del modelo Exp016. La línea discontinua indica el umbral de éxito de 0,4 m. Todos los fallos corresponden a truncamientos por límite de pasos, sin terminaciones de seguridad; una parte de ellos queda próxima al umbral, aunque existen episodios con distancias finales superiores. .	50
A.1. Ejemplo de métricas de <i>rollout</i> de TensorBoard para Exp010. La captura ilustra el formato de consulta de los escalares registrados durante el entrenamiento; la interpretación del experimento se realiza junto con los monitores de episodio y la matriz de resultados.	64

Índice de Tablas

2.1. Relación entre las capas tecnológicas empleadas en el presente trabajo.	9
2.2. Diferencias principales entre aprendizaje profundo supervisado y aprendizaje por refuerzo.	10
3.1. Resumen del contrato principal del entorno Gymnasium.	23
3.2. Estados y fallos considerados en el diseño del reinicio de episodios.	24
3.3. Componentes de la función de recompensa y condiciones de finalización del entorno.	26
4.1. Organización principal del software implementado.	30
4.2. Principales parámetros PPO definidos en la configuración base de entrenamiento.	33
4.3. Herramientas principales utilizadas durante la implementación y validación.	35
5.1. Niveles de validación aplicados al entorno desarrollado.	40
5.2. Validadores representativos utilizados durante el desarrollo.	41
5.3. Fallos de integración identificados y mecanismos de control incorporados.	43
6.1. Resumen de experimentos principales y su interpretación.	46
6.2. Comparación de controladores sobre los mismos 100 episodios sembrados. Pasos y tiempo corresponden a la mediana de los episodios con éxito; el error de yaw es la media absoluta en el instante terminal.	49
A.1. Configuraciones representativas de la fase de estabilización.	62
A.2. Rutas de consulta de configuraciones, scripts y artefactos experimentales en el repositorio.	63
A.3. Resumen del uso de herramientas de inteligencia artificial generativa.	65

Índice de Listings

A.1. Patrón general de entrenamiento PPO a partir de una configuración YAML.	62
A.2. Patrón general de entrenamiento supervisado con reanudación.	63
A.3. Patrón general de evaluación por semillas.	63
A.4. Consulta de métricas TensorBoard para una ejecución concreta.	64

Resumen

Este trabajo desarrolla un entorno de aprendizaje por refuerzo para drones sobre el simulador dinámico de multirrotores de Aerostack2, un *framework* de robótica aérea basado en ROS 2. Sobre este entorno se entrena y evalúa un controlador de velocidad de alto nivel. La tarea consiste en alcanzar objetivos con posición y orientación aleatorias desde estados iniciales también aleatorios, mediante observaciones relativas normalizadas y comandos continuos de velocidad lineal y de guiñada.

La primera contribución es la infraestructura experimental. Se implementa un entorno Gymnasium integrado con Aerostack2, un reinicio determinista del simulador con certificación de accionabilidad, guards predictivos de acción, un supervisor para campañas largas con reanudación desde *checkpoint* y reciclado preventivo, y ejecución vectorizada de dos drones en un mismo simulador. En conjunto, estos componentes permiten iniciar episodios solo cuando el vehículo responde físicamente a los comandos, corregir aproximaciones a fronteras no observables y duplicar el muestreo efectivo.

La segunda contribución es la resolución de la tarea de navegación. Con una recompensa mínima basada en distancia y un curriculum de tres etapas que amplía progresivamente el rango de distancias, la tasa de éxito pasa de valores inferiores al 1 % al 100 % en una evaluación determinista de 100 episodios reproducibles por semillas.

La evaluación final compara la política aprendida con un controlador PID de referencia sobre los mismos episodios. Ambos alcanzan el 100 % de éxito, pero el PID resuelve la tarea nominal con mayor rapidez y mejor precisión de orientación. Este resultado es coherente con la naturaleza del problema: el control de velocidad hacia un objetivo estático favorece a una ley proporcional. Por ello, el valor del enfoque aprendido no reside en superar al PID en este caso concreto, sino en alcanzar una paridad funcional y en proporcionar una infraestructura aplicable a tareas sin una ley de control evidente.

El trabajo también identifica dos limitaciones de ingeniería del simulador: la degradación acumulativa asociada a los reinicios de servicio y la persistencia indefinida de las referencias de velocidad. Estos hallazgos motivan, como línea futura principal, el entrenamiento sobre *bindings* directos de la dinámica sin ROS 2 en el bucle.

Palabras Clave: aprendizaje por refuerzo, vehículos aéreos no tripulados, Aerostack2, ROS 2, PPO, curriculum, simulación, control de velocidad.

Abstract

This work develops a reinforcement learning environment for drones on top of the Aerostack2 multirotor dynamics simulator, a ROS 2 based aerial robotics framework. The environment is used to train and evaluate a high-level velocity controller. The task consists of reaching targets with random positions and headings from random initial states, using normalized relative observations and continuous linear and yaw velocity commands.

The first contribution is the experimental infrastructure. The work implements a Gymnasium environment integrated with Aerostack2, a deterministic simulator reset with actionability certification, predictive action guards, a supervisor for long training campaigns with checkpoint resumption and preventive recycling, and vectorized execution of two drones in a single simulator. Together, these components make it possible to start episodes only when the vehicle physically responds to commands, correct approaches to unobservable boundaries, and double the effective sampling rate.

The second contribution is solving the navigation task. With a minimal distance-based reward and a three-stage curriculum that progressively expands the distance range, the success rate increases from below 1 % to 100 % in a deterministic evaluation of 100 seed-reproducible episodes.

The final evaluation compares the learned policy with a reference PID controller on the same episodes. Both reach 100 % success, but the PID solves the nominal task faster and with better final heading accuracy. This result is consistent with the nature of the problem: velocity control towards a static target favors a proportional law. Therefore, the value of the learned approach does not lie in outperforming the PID in this specific case, but in achieving functional parity and providing an infrastructure that can be applied to tasks without an obvious control law.

The work also identifies two engineering limitations of the simulator: cumulative degradation associated with reset service calls and the indefinite persistence of velocity references. These findings motivate, as the main future line, training over direct dynamics bindings without ROS 2 in the loop.

Keywords: reinforcement learning, unmanned aerial vehicles, Aerostack2, ROS 2, PPO, curriculum learning, simulation, velocity control.

Capítulo 1

Introducción

Este capítulo presenta el marco general del Trabajo Fin de Máster, centrado en la aplicación de aprendizaje por refuerzo al control de vehículos aéreos no tripulados dentro del ecosistema Aerostack2. En primer lugar se expone la motivación del trabajo y su relevancia en el contexto actual de la robótica aérea autónoma. A continuación se describe el contexto técnico y de investigación en el que se enmarca el desarrollo realizado. Posteriormente se definen el objetivo general y los objetivos específicos. Por último, se resume la organización de la memoria.

1.1. Motivación del proyecto

Los vehículos aéreos no tripulados han pasado en pocos años de ser una tecnología principalmente experimental a convertirse en una herramienta útil en aplicaciones industriales, logísticas, de inspección y competición. En estos escenarios, la autonomía y la capacidad de respuesta del vehículo son aspectos críticos, especialmente cuando el dron debe seguir referencias espaciales, reaccionar ante cambios del entorno o mantener un comportamiento estable en maniobras rápidas.

Dentro de este contexto, el control clásico basado en reguladores PID sigue siendo una solución ampliamente utilizada por su simplicidad, interpretabilidad y robustez. Sin embargo, cuando la dinámica del sistema es no lineal, el régimen de operación es exigente o se desea adaptar el comportamiento a criterios de tarea más amplios, el ajuste manual de estos controladores puede resultar costoso y difícil de generalizar. Frente a ello, el aprendizaje por refuerzo ofrece una alternativa de interés, permite que una política de control aprenda a actuar a partir de la interacción con un entorno, optimizando un criterio de comportamiento definido mediante recompensas.

La motivación de este trabajo surge en la intersección entre robótica aérea e inteligencia artificial. En el grupo CVAR (*Computer Vision and Aerial Robotics*) existe interés en desarrollar soluciones de control cada vez más ágiles para drones, en particular en el marco de la competición A2RL, donde el rendimiento del vehículo depende de su capacidad para ejecutar maniobras rápidas y precisas. Para que una aproximación basada en aprendizaje por refuerzo sea viable, no basta con seleccionar un algoritmo de entrenamiento sino que resulta imprescindible disponer de una infraestructura experimental que conecte de forma correcta el simulador, el *middleware* robótico y las herramientas de aprendizaje automático.

Este Trabajo Fin de Máster responde a esa necesidad mediante la construcción de un entorno reproducible que integra Gymnasium, Stable-Baselines3, ROS 2 y Aerostack2. La aportación principal no se formula todavía como una demostración de superioridad del aprendizaje por refuerzo frente a un controlador clásico, sino como la base técnica necesaria para entrenar, diagnosticar y evaluar políticas de control en condiciones controladas y extensibles. En consecuencia, el alcance del trabajo prioriza la reproducibilidad de la infraestructura, la validez del ciclo de episodio y la disponibilidad de diagnósticos verificables frente a afirmaciones de aprendizaje robusto que no estén respaldadas por experimentos completos.

1.2. Contexto del proyecto

El trabajo se desarrolla sobre Aerostack2, un *framework* para robótica aérea basado en ROS 2 que proporciona componentes reutilizables para control, estimación, comportamientos de vuelo e integración con diferentes plataformas y simuladores. En particular, se utiliza el simulador dinámico de multirrotores de Aerostack2, una herramienta sin interfaz gráfica que modela la dinámica del vehículo y que puede visualizarse mediante RViz2. Al no requerir interfaz gráfica, resulta adecuado para plantear experimentos con varios drones dentro de un mismo simulador compartido, posibilidad que este trabajo valida experimentalmente con dos drones entrenando en paralelo.

El repositorio `rl_uav_aerostack2` contiene el entorno `AS2TestEnv-v0`, registrado en `rl_uav/__init__.py` e implementado en `rl_uav/envs/as2_test_env.py`. Este entorno encapsula el ciclo básico de Gymnasium, inicializa la interfaz `DroneInterface` de Aerostack2, arma el dron, activa el modo *offboard*, ejecuta el despegue y envía comandos de velocidad mediante Aerostack2 y ROS 2. La acción del agente es un vector $[v_x, v_y, v_z, v_{yaw}]$, compuesto por velocidades lineales y una velocidad angular en yaw.

La observación principal del entorno no está formada por posición y velocidad absolutas, sino por el vector normalizado $[dx, dy, dz, d_{yaw}]$, que representa la diferencia de posición y orientación respecto al objetivo. La recompensa empleada en la campaña principal es deliberadamente mínima: una penalización continua por distancia al objetivo y una bonificación terminal de éxito reducida según el error de orientación final. La finalización del episodio contempla el éxito al alcanzar una distancia inferior al umbral definido, las terminaciones de seguridad por salida de límites o baja altitud, y el truncado por superar el número máximo de pasos.

Sobre esta base se ha construido la infraestructura completa de entrenamiento y evaluación con PPO mediante Stable-Baselines3: configuraciones declarativas por experimento, registro de métricas y *checkpoints*, supervisión de campañas largas con reanudación automática, entrenamiento vectorizado con varios drones sobre un mismo simulador y un protocolo de evaluación reproducible por semillas. Con esa infraestructura, la política final resuelve la tarea completa con un 100 % de éxito en evaluación determinista y se compara, sobre episodios idénticos, con un controlador PID de referencia.

1.3. Objetivos

El objetivo general de este Trabajo Fin de Máster es desarrollar una infraestructura reproducible para formular, entrenar y evaluar controladores de alto nivel para drones mediante aprendizaje por refuerzo en un entorno de simulación integrado con un *stack* robótico.

Para alcanzar este objetivo general, el trabajo se organiza en los siguientes objetivos específicos:

- Analizar el problema de control de alto nivel de un dron en simulación, identificando las variables de observación, las acciones de velocidad y las condiciones de terminación necesarias para formularlo como un entorno de aprendizaje por refuerzo.
- Diseñar e implementar un entorno de aprendizaje que integre el simulador dinámico con el *middleware* robótico y permita ejecutar el ciclo de interacción entre agente y dron de forma reproducible.
- Incorporar una función de recompensa coherente con el seguimiento de objetivos espaciales y la orientación final del vehículo, manteniendo información de diagnóstico suficiente para analizar el comportamiento del entorno.
- Entrenar una política capaz de resolver la tarea de navegación con inicio, objetivo y orientación aleatorios, empleando la infraestructura de entrenamiento con PPO y la vectorización de múltiples drones dentro de un simulador compartido.
- Analizar las limitaciones de integración observadas durante los experimentos, especialmente las asociadas al reinicio de episodios, la accionabilidad de los comandos y la reproducibilidad física del simulador.
- Evaluar el controlador aprendido con un protocolo reproducible y compararlo con un controlador PID de referencia sobre episodios idénticos.

1.4. Estructura del documento

La memoria se organiza siguiendo una progresión desde el contexto técnico hasta la evaluación del sistema construido. Tras esta introducción, el Capítulo 2 presenta el estado del arte y el trabajo relacionado sobre marcos software para robótica aérea, aprendizaje por refuerzo aplicado a UAV y simuladores de entrenamiento. El Capítulo 3 desarrolla el diseño de la solución, incluyendo la formulación del entorno, las decisiones de observación, acción, recompensa y terminación, y la estrategia de integración con Aerostack2.

El Capítulo 4 describe la implementación realizada, con atención a la estructura del repositorio, la conexión con ROS 2, la infraestructura PPO y el soporte de vectorización. El Capítulo 5 recoge las pruebas de validación funcional de la infraestructura y los criterios usados para comprobar que el entorno se comporta de forma consistente. El Capítulo 6 presenta los resultados disponibles y su discusión, distinguiendo entre validaciones de infraestructura y resultados de aprendizaje que requieran experimentación posterior. El Capítulo 7 analiza el impacto del trabajo desde una perspectiva técnica y académica. Finalmente, el Capítulo 8 expone las conclusiones, revisa el grado de cumplimiento de los objetivos y plantea líneas futuras de desarrollo.

Capítulo 2

Fundamentos técnicos y trabajo relacionado

Este capítulo introduce los fundamentos técnicos necesarios para comprender el entorno desarrollado y revisa los trabajos relacionados con aprendizaje por refuerzo aplicado a vehículos aéreos no tripulados. No se pretende realizar una revisión exhaustiva de todos los conceptos de robótica o aprendizaje automático, sino seleccionar aquellos elementos que condicionan directamente el diseño posterior como el middleware robótico empleado, la arquitectura aérea sobre la que se ejecuta el sistema, el papel de la simulación y la formulación del problema de aprendizaje.

La organización sigue una progresión de infraestructura a método. En primer lugar, se presentan ROS 2 y Aerostack2 como base software para construir sistemas aéreos autónomos. A continuación, se revisa el aprendizaje por refuerzo profundo en drones y se justifica el uso de PPO como algoritmo de entrenamiento. Después se analizan simuladores y entornos de entrenamiento relevantes, con especial atención a las implicaciones de usar un simulador integrado en un *stack* robótico. Finalmente, se sintetiza el posicionamiento del presente trabajo respecto a la literatura revisada.

2.1. Infraestructura robótica: ROS 2 y Aerostack2

El desarrollo de sistemas aéreos autónomos no consiste únicamente en programar un algoritmo de control. Un UAV operativo requiere coordinar estimación de estado, control, comunicaciones, gestión de modos de vuelo, seguridad, planificación y, en muchos casos, percepción. Esta integración es uno de los principales motivos por los que los trabajos de robótica suelen apoyarse en arquitecturas software reutilizables que permiten que los algoritmos se conecten con un sistema completo en lugar de ejecutarse como componentes aislados.

2.1.1. ROS 2 como middleware robótico

ROS 2 proporciona una infraestructura de comunicación para sistemas robóticos distribuidos. Un sistema ROS 2 se organiza como un grafo de nodos que intercambian información mediante distintos mecanismos de comunicación. Los *topics* permiten publicar flujos continuos de datos, los servicios modelan interacciones de petición y respuesta, y las acciones representan tareas de mayor duración con realimentación

y posibilidad de cancelación [Open Robotics, 2026]. Esta separación resulta especialmente útil en robótica aérea, donde conviven señales continuas de estado con órdenes discretas de ciclo de vida, como armar, despegar o aterrizar.

La Figura 2.1 muestra esta idea de forma conceptual. Un nodo puede actuar como publicador o suscriptor de un *topic*, lo que permite distribuir información de estado o medidas de sensores. El mismo nodo también puede ofrecer o consumir servicios, adecuados para operaciones puntuales que requieren una respuesta explícita. Esta distinción es importante en el presente trabajo porque el ciclo de aprendizaje combina ambos tipos de interacción: lectura recurrente del estado del UAV y ejecución de órdenes de preparación o control sobre la plataforma simulada.

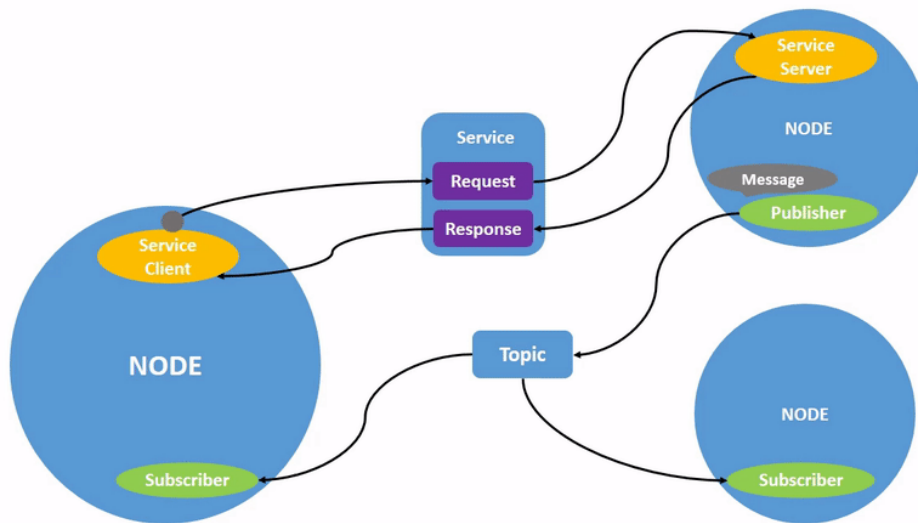


Figura 2.1: Comunicación entre nodos en ROS mediante *topics* y servicios. Fuente: Chollet [Chollet, 2021].

Desde el punto de vista de este trabajo, ROS 2 se usa como la capa que permite comunicar el entorno de aprendizaje con el sistema robótico simulado. La política de aprendizaje por refuerzo no publica directamente en todos los *topics* ni gestiona todos los servicios disponibles; esa responsabilidad queda encapsulada en las capas de Aerostack2 y en la interfaz desarrollada. Esta decisión evita que el algoritmo de aprendizaje dependa de detalles de comunicación de bajo nivel.

2.1.2. Aerostack2

Sobre esta base se sitúa Aerostack2. Inicialmente Aerostack fue propuesto como un marco software abierto, modular y multicapa para el desarrollo de sistemas aéreos autónomos complejos [Sánchez-López et al., 2017]. Su objetivo era proporcionar una arquitectura reutilizable capaz de integrar componentes de control, percepción, planificación y ejecución de misiones en robótica aérea. Esta idea es relevante porque desplaza el foco desde el algoritmo aislado hacia el sistema completo que permite ejecutar una misión aérea.

Por otra parte, Aerostack2 constituye la evolución de esta línea sobre ROS 2. Su

2.1. Infraestructura robótica: ROS 2 y Aerostack2

diseño enfatiza la independencia de plataforma, la arquitectura basada en *plugins*, el soporte para sistemas multirrobot y la definición de misiones mediante comportamientos reutilizables [Fernández-Cortizas et al., 2023]. En lugar de imponer un único simulador o una única plataforma física, Aerostack2 actúa como capa de abstracción para conectar distintos vehículos, simuladores y componentes de misión. Esta característica lo hace adecuado para trabajos en los que interesa aproximar la experimentación en simulación a un flujo de robótica aérea operativo.

La arquitectura de Aerostack2 puede entenderse como una organización por capas, resumida en la Figura 2.2. En los niveles inferiores se sitúan el middleware, los mensajes de comunicación y las interfaces con plataformas o sensores. Sobre ellos se construyen funciones robóticas básicas, como estimación de estado, control de movimiento o gestión de emergencias. A un nivel superior aparecen los comportamientos de robot, como despegue, aterrizaje, seguimiento de trayectorias o detección de marcadores. Finalmente, las capas de misión y aplicación permiten componer esas capacidades para casos de uso concretos.

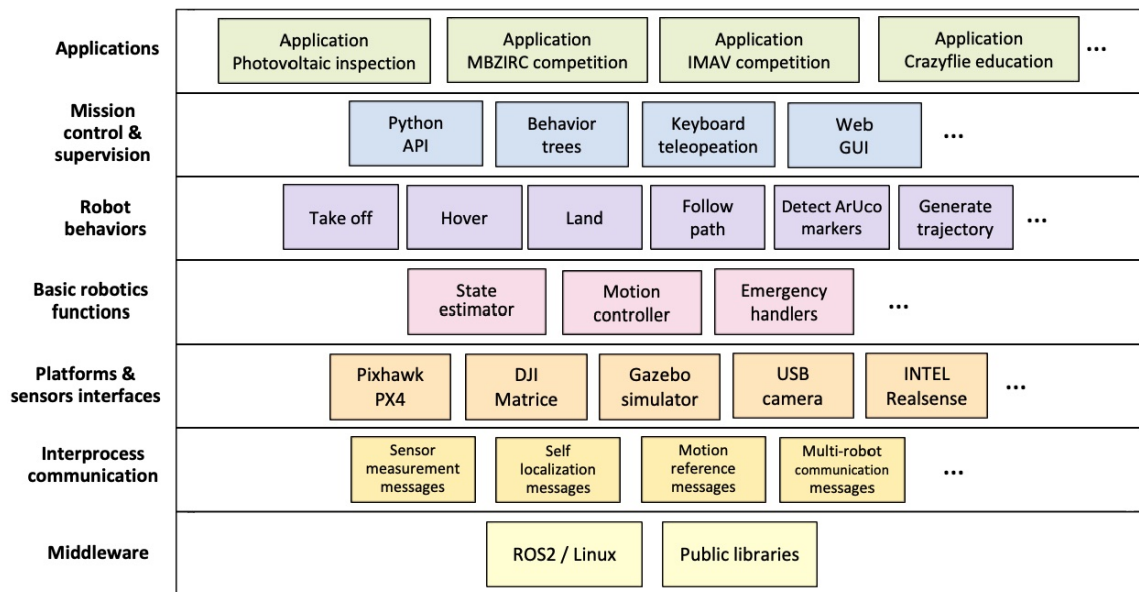


Figura 2.2: Organización por capas de los componentes software de Aerostack2. Fuente: Fernández-Cortizas et al. [Fernández-Cortizas et al., 2023].

Esta organización por capas ayuda a explicar por qué Aerostack2 resulta especialmente útil para el presente trabajo. El entorno de aprendizaje por refuerzo no sustituye a los controladores, estimadores o comportamientos de Aerostack2, sino que se conecta por encima de ellos para emitir consignas de velocidad dentro de una infraestructura robótica ya existente. Así, el problema de aprendizaje se mantiene centrado en decidir acciones de alto nivel, mientras que la ejecución robótica queda delegada en componentes especializados del *stack*.

Nivel	Función principal	Papel en este trabajo
ROS 2	Comunicación entre nodos mediante <i>topics</i> , servicios y acciones.	Transporte de órdenes, estados y operaciones de ciclo de vida.
Aerostack2	Arquitectura aérea sobre ROS 2 con interfaces y comportamientos reutilizables.	Sustrato robótico usado por el entorno de aprendizaje.
Simulador AS2	Ejecución simulada del multirrotor y de sus interfaces robóticas.	Fuente de experiencia para entrenamiento y validación funcional.
Gymnasium/SB3	Interfaz estándar de aprendizaje por refuerzo y entrenamiento PPO.	Capa de interacción del agente con observaciones, acciones y recompensas.

Cuadro 2.1: Relación entre las capas tecnológicas empleadas en el presente trabajo.

El Cuadro 2.1 resume la función de cada capa. Esta separación ayuda a delimitar la contribución del trabajo. No se propone un nuevo middleware robótico ni una nueva arquitectura aérea, sino una interfaz reproducible entre un entorno de aprendizaje por refuerzo y una plataforma aérea basada en ROS 2 y Aerostack2. Por ello, los capítulos posteriores distinguen entre la infraestructura robótica reutilizada, el diseño del problema de aprendizaje y la implementación del entorno Gymnasium.

2.1.3. Gymnasium y Stable-Baselines3 en el flujo de aprendizaje

Gymnasium proporciona una API común para definir entornos de aprendizaje por refuerzo mediante operaciones como `reset` y `step`, junto con espacios de observación y acción explícitos [Towers et al., 2024; Farama Foundation, 2026]. Su papel en este trabajo no es simular por sí mismo la dinámica del UAV, sino ofrecer el contrato mediante el cual un algoritmo de aprendizaje puede interactuar de forma uniforme con una tarea implementada sobre ROS 2 y Aerostack2. Esta separación permite que la lógica del agente no dependa directamente de los detalles de comunicación robótica.

Stable-Baselines3 aporta implementaciones mantenidas de algoritmos de aprendizaje por refuerzo profundo y utilidades de entrenamiento compatibles con este tipo de entornos, incluida la implementación de PPO utilizada más adelante [Stable-Baselines3 Contributors, 2026]. En el presente trabajo se emplea como capa de entrenamiento y monitorización, mientras que la definición del problema, la recompensa, los reinicios y las condiciones de seguridad permanecen en el entorno desarrollado. Por tanto, Gymnasium estandariza la interfaz y Stable-Baselines3 proporciona el algoritmo, pero la validez experimental depende de que la adaptación robótica produzca transiciones físicas fiables.

2.1.3.0.1. Herramientas de observación y monitorización. En una infraestructura de aprendizaje por refuerzo integrada con ROS 2, la observación del sistema no se limita a las métricas internas del algoritmo. RViz2 proporciona una visualización espacial del estado del UAV, de sus referencias y de los marcadores publicados en el grafo ROS 2, por lo que resulta útil para validar cualitativamente que la escena simulada y los comandos robóticos son coherentes. TensorBoard, por su parte, permite inspeccionar las métricas emitidas por Stable-Baselines3 y por los monitores de

episodio, como recompensa media, longitud de episodio y señales auxiliares registradas durante la ejecución. En este trabajo, ambas herramientas se emplean como soporte de validación y diagnóstico, no como sustituto de las métricas cuantitativas del entorno.

2.2. Aprendizaje por refuerzo para el control de UAV

El aprendizaje por refuerzo se diferencia de otros enfoques de aprendizaje automático porque el conocimiento no se obtiene a partir de un conjunto fijo de pares entrada-salida, sino mediante interacción secuencial con un entorno. En aprendizaje supervisado clásico, una red neuronal aprende a aproximar una función usando ejemplos etiquetados; en aprendizaje por refuerzo, en cambio, el agente debe actuar, observar las consecuencias de sus acciones y ajustar su comportamiento según una señal de recompensa. Esta diferencia es esencial en robótica: un robot no solo debe reconocer patrones, sino decidir acciones que modifican el estado físico del sistema.

Esta diferencia puede resumirse en el Cuadro 2.2. En aprendizaje profundo supervisado, el objetivo habitual es aprender una relación estática entre entradas y etiquetas. En aprendizaje por refuerzo, el problema es dinámico las decisiones modifican los datos que se observarán después. Por ello, la calidad del aprendizaje no depende solo del modelo neuronal, sino también de la definición del entorno, de la recompensa y de la estrategia de exploración [Sutton y Barto, 2018; Molina, 2025b].

Aspecto	Aprendizaje profundo supervisado	Aprendizaje por refuerzo
Datos de entrenamiento	Conjunto de ejemplos normalmente fijo y etiquetado.	Experiencia generada por interacción con el entorno.
Señal de aprendizaje	Error entre predicción y etiqueta conocida.	Recompensa escalar y acumulativa.
Salida del modelo	Predicción, clasificación o regresión.	Acción o distribución de acciones.
Dependencia temporal	Las muestras suelen tratarse como ejemplos independientes.	Cada acción condiciona los estados y recompensas futuros.
Relevancia en robótica	Percepción, estimación o reconocimiento.	Control, navegación y toma de decisiones secuencial.

Cuadro 2.2: Diferencias principales entre aprendizaje profundo supervisado y aprendizaje por refuerzo.

La Figura 2.3 complementa esta comparación. El aprendizaje por refuerzo clásico puede representarse mediante tablas de valor cuando los estados y acciones son discretos y de tamaño limitado. El aprendizaje profundo supervisado, por su parte, utiliza redes neuronales para extraer patrones a partir de datos etiquetados. El aprendizaje por refuerzo profundo combina ambas ideas: emplea redes neuronales para aproximar políticas, funciones de valor o ambas, pero mantiene el ciclo de interacción con el entorno propio del aprendizaje por refuerzo. Esta combinación es la que permite aplicar RL a problemas con estados continuos, observaciones de alta dimensión o espacios de acción complejos, como los que aparecen en robótica aérea.

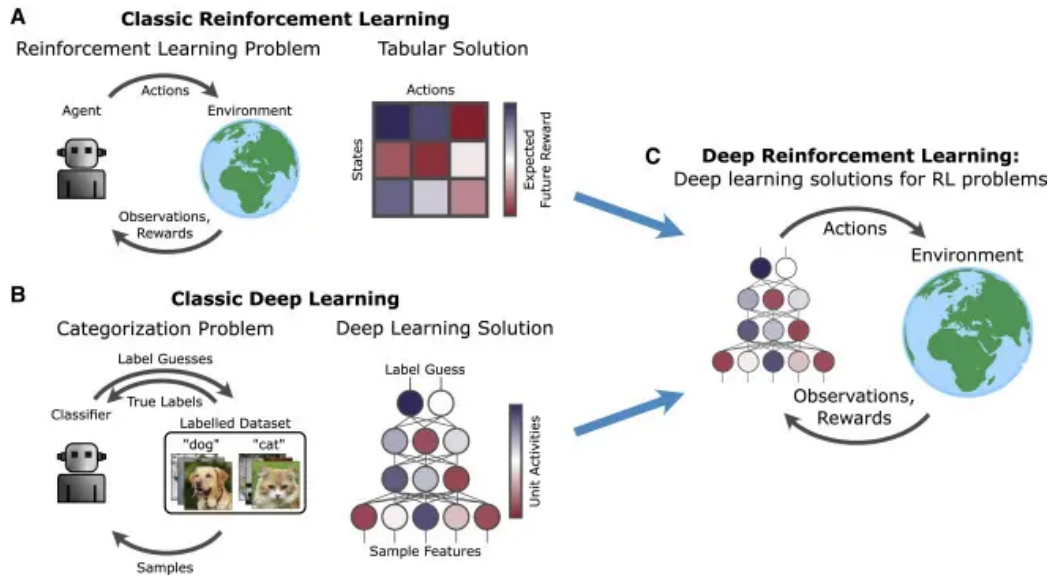


Figura 2.3: Relación conceptual entre aprendizaje por refuerzo clásico, aprendizaje profundo supervisado y aprendizaje por refuerzo profundo. Fuente: Objectways [Objectways, 2025].

La Figura 2.4 resume el ciclo básico de aprendizaje por refuerzo. En cada instante t , el agente recibe un estado u observación S_t , selecciona una acción A_t y el entorno responde con una nueva situación S_{t+1} y una recompensa R_{t+1} [Sutton y Barto, 2018]. En un problema robótico, el entorno incluye tanto la dinámica del vehículo como la infraestructura que ejecuta las órdenes y proporciona medidas del estado. En este trabajo, ese papel lo desempeñan conjuntamente Aerostack2, ROS 2 y el simulador multirrotor.

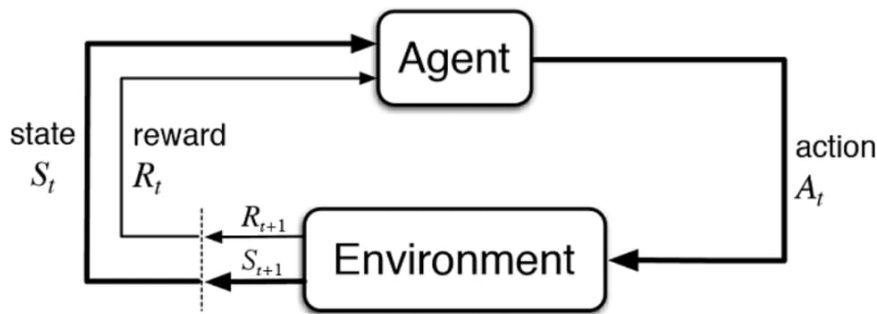


Figura 2.4: Bucle de interacción entre agente y entorno en aprendizaje por refuerzo. Fuente: AltexSoft [AltexSoft, 2021], basado en la formulación clásica de Sutton y Barto [Sutton y Barto, 2018].

Formalmente, un problema de aprendizaje por refuerzo suele expresarse como un proceso de decisión de Markov. Esta formulación se define mediante un conjunto de estados S , un conjunto de acciones A , una función de transición P , una función de recompensa R y un factor de descuento $\gamma \in [0, 1]$ [Sutton y Barto, 2018]. La política π determina cómo actúa el agente: puede representarse como una función determinista

$\pi(s) = a$ o como una distribución de probabilidad $\pi(a|s)$ sobre acciones posibles. El objetivo del aprendizaje consiste en encontrar una política que maximice la recompensa acumulada esperada.

El retorno descontado desde un instante t se define como:

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1}.$$

Este valor resume la utilidad futura de una secuencia de decisiones. El factor de descuento γ permite ponderar las recompensas lejanas: valores cercanos a uno consideran horizontes más largos, mientras que valores menores favorecen recompensas inmediatas. En robótica aérea esta distinción es importante, porque una acción puede parecer buena a corto plazo, por ejemplo aumentar la velocidad hacia el objetivo, pero generar trayectorias inestables o salidas de la región segura en pasos posteriores.

A partir del retorno se definen dos funciones habituales. La función de valor $V^\pi(s)$ estima el retorno esperado al seguir la política π desde un estado s . La función de valor-acción $Q^\pi(s, a)$ estima el retorno esperado al ejecutar una acción concreta a en el estado s y continuar después con la política π . La diferencia entre ambas magnitudes permite definir la ventaja:

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s).$$

La ventaja mide si una acción concreta es mejor o peor que el comportamiento medio esperado en ese estado. Esta idea será relevante para PPO, ya que el algoritmo usa estimaciones de ventaja para decidir en qué dirección debe modificarse la política.

En el contexto de vehículos aéreos no tripulados, el aprendizaje por refuerzo resulta atractivo porque permite optimizar comportamientos difíciles de programar mediante reglas cerradas. Las revisiones recientes muestran aplicaciones en planificación, navegación y control de UAV, especialmente en escenarios con dinámica no lineal, espacios de acción continuos o transferencia desde simulación [Azar et al., 2021; Chan et al., 2024]. Sin embargo, estas mismas revisiones subrayan limitaciones importantes: necesidad de muchas interacciones, sensibilidad al diseño de la recompensa, dependencia de la fidelidad del simulador y dificultad de trasladar políticas entrenadas a plataformas reales.

Una parte significativa de los trabajos se ha centrado en control a bajo nivel, es decir, en aprender políticas que actúan cerca de los actuadores o de las consignas de actitud. Pi et al. propusieron un controlador de bajo nivel basado en aprendizaje por refuerzo capaz de generalizar a vehículos multirrotor con distintas configuraciones físicas [Pi et al., 2021]. En una línea relacionada, Yoo et al. combinaron controladores deterministas clásicos con una política aprendida, mostrando que una aproximación híbrida puede mejorar la convergencia y el seguimiento de trayectorias mediante control en velocidad [Yoo et al., 2021]. Estos trabajos son relevantes para una posible extensión jerárquica del presente trabajo, aunque el entorno desarrollado se sitúa actualmente en un nivel de control más alto: la política genera comandos de velocidad lineal y velocidad angular en yaw, no señales de motor ni tasas angulares completas.

Otra línea activa es la del vuelo agresivo y el empuje del dron hacia sus límites dinámicos. Kaufmann et al. demostraron en *Deep Drone Acrobatics* que es posible entrenar

en simulación una política para realizar maniobras acrobáticas y transferirla al dron real sin ajuste fino posterior [Kaufmann et al., 2020]. Más adelante, los mismos autores analizaron comparativamente distintas abstracciones de control y observaron que las políticas que actúan sobre *body-rates* y *thrust* resultan más robustas en transferencia que aquellas que operan directamente sobre los motores [Kaufmann et al., 2022]. Este punto muestra que el nivel de abstracción de la acción no es una decisión secundaria, sino un elemento central del diseño del problema de aprendizaje.

Los resultados recientes en vuelo agresivo refuerzan esta idea. Eschmann et al. mostraron que, con una arquitectura adecuada y un simulador muy optimizado, es posible entrenar controladores de bajo nivel en tiempos extraordinariamente reducidos y mantener capacidad de transferencia a plataformas reales [Eschmann et al., 2024]. En paralelo, Song et al. estudiaron de manera sistemática la competición entre control óptimo y aprendizaje por refuerzo en un problema de carrera autónoma con drones, concluyendo que el aprendizaje por refuerzo puede superar al control óptimo cuando el objetivo se formula directamente a nivel de tarea y se entrena con suficiente robustez frente a incertidumbres del modelo [Song et al., 2023]. Este resultado es pertinente para el contexto de A2RL que motiva el trabajo, aunque debe interpretarse como antecedente de la literatura y no como evidencia del rendimiento del entorno aquí desarrollado.

La literatura revisada revela, por tanto, una diversidad de formulaciones. Muchos trabajos se centran en estabilización de actitud, control a nivel de actuadores, vuelo acrobático o navegación *end-to-end* basada en percepción [Kaufmann et al., 2020; Pi et al., 2021; Yoo et al., 2021]. A partir de esas obras puede inferirse que existe menos atención a controladores intermedios integrados en un *stack* robótico de ROS 2, donde la acción se expresa como $[v_x, v_y, v_z, v_{yaw}]$ y la observación principal como un estado relativo normalizado $[dx, dy, dz, d_{yaw}]$. Esta inferencia ayuda a delimitar el espacio de contribución del presente trabajo sin convertirlo en una afirmación absoluta sobre toda la literatura existente.

2.2.1. PPO como algoritmo de entrenamiento

PPO, *Proximal Policy Optimization*, pertenece a la familia de métodos de gradiente de política. Estos métodos no aprenden únicamente una función que valora acciones, sino que optimizan directamente una política parametrizada $\pi_\theta(a|s)$ [Schulman et al., 2017]. En aprendizaje por refuerzo profundo, esta política se implementa habitualmente mediante una red neuronal que recibe una observación y produce una distribución de acciones. En espacios continuos, como el empleado en este trabajo, la acción puede interpretarse como una muestra o una salida de una distribución sobre velocidades admisibles.

PPO suele implementarse con una arquitectura actor-crítico. El actor representa la política y decide qué acción ejecutar. El crítico estima el valor de los estados o de las acciones y proporciona una señal auxiliar para evaluar si la acción tomada fue mejor o peor de lo esperado. La Figura 2.5 muestra este esquema: las trayectorias generadas por la interacción con el entorno se usan para actualizar tanto el crítico como la política del actor. Esta separación es útil porque el actor aprende a decidir, mientras que el crítico reduce la varianza de la estimación del gradiente mediante información de valor.

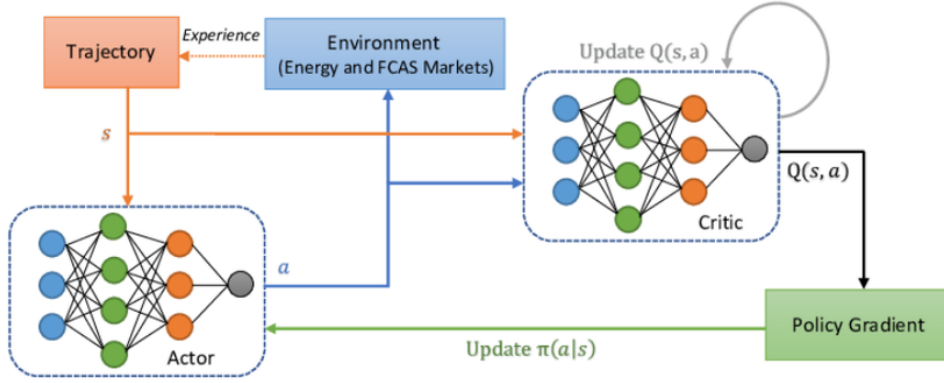


Figura 2.5: Arquitectura actor-crítico utilizada habitualmente en PPO. Fuente: ResearchGate [ResearchGate, 2024].

En una arquitectura actor-crítico, el crítico permite estimar la ventaja sin esperar siempre al retorno completo de un episodio. Una aproximación habitual es partir del error temporal:

$$\delta_t = R_{t+1} + \gamma V(s_{t+1}) - V(s_t),$$

que compara la recompensa inmediata más el valor estimado del estado siguiente con el valor estimado del estado actual. En implementaciones modernas de PPO se emplea frecuentemente una estimación de ventaja generalizada, que combina varios errores temporales ponderados:

$$\hat{A}_t = \sum_{l=0}^{T-t-1} (\gamma\lambda)^l \delta_{t+l}.$$

El parámetro λ controla el compromiso entre sesgo y varianza: valores altos incorporan más información de retornos largos, mientras que valores bajos se apoyan más en el valor estimado por el crítico. Esta explicación es relevante para interpretar parámetros como `gamma` y `gae_lambda` en la configuración de entrenamiento del presente trabajo [Molina, 2025a; Molina, 2025c].

La dificultad principal de los métodos de gradiente de política es que una actualización demasiado grande puede destruir una política que estaba funcionando razonablemente bien. PPO aborda este problema comparando la política nueva con la política anterior mediante la razón de probabilidades:

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}.$$

Si esta razón se aleja demasiado de uno, significa que la nueva política asigna una probabilidad muy distinta a la acción tomada respecto a la política que generó los datos. PPO limita ese cambio mediante una función objetivo recortada:

$$L^{CLIP}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right].$$

El parámetro ϵ define cuánto puede separarse la nueva política de la anterior en una actualización. Si la ventaja estimada $\hat{A}_t$ es positiva, la actualización aumenta la probabilidad de la acción, pero solo hasta el límite permitido por el recorte. Si la ventaja

es negativa, reduce esa probabilidad, de nuevo sin permitir cambios excesivos. Este mecanismo proporciona una forma práctica de mantener las actualizaciones cerca de la política que generó las muestras, evitando saltos demasiado bruscos [Schulman et al., 2017].

El funcionamiento práctico de PPO puede resumirse como un ciclo de cinco pasos. Primero, la política actual interactúa con el entorno y genera trayectorias de estados, acciones, recompensas y finales de episodio. Segundo, el crítico estima valores y se calculan ventajas para cada transición. Tercero, se actualiza el actor maximizando la función objetivo recortada. Cuarto, se entrena el crítico para reducir el error entre sus valores estimados y los retornos objetivo. Quinto, se puede añadir un término de entropía para favorecer exploración y evitar que la política se vuelva determinista demasiado pronto. En la práctica, muchas implementaciones optimizan una combinación de pérdida de política, pérdida de valor y bonificación de entropía [Schulman et al., 2017; Stable-Baselines3 Contributors, 2026]:

$$L(\theta) = L^{CLIP}(\theta) - c_1 L^{VF}(\theta) + c_2 S[\pi_\theta].$$

Aquí, L^{VF} representa el error de la función de valor, $S[\pi_\theta]$ la entropía de la política, y c_1 y c_2 ponderan la importancia relativa de ambos términos. Esta expresión permite entender por qué la configuración de PPO incluye parámetros específicos para la función de valor, la exploración y la norma del gradiente, no se entrena solo una red que “elige acciones”, sino un sistema de optimización que equilibra mejora de política, estimación de valor y estabilidad.

En el presente trabajo, PPO resulta adecuado por tres motivos. En primer lugar, el espacio de acciones es continuo y acotado, ya que la política genera velocidades lineales y velocidad de guiñada. En segundo lugar, su implementación en Stable-Baselines3 se integra de forma directa con entornos Gymnasium y entornos vectorizados. En tercer lugar, el mecanismo de *clipping* reduce el riesgo de actualizaciones inestables, un aspecto relevante cuando cada muestra requiere interactuar con un simulador robótico [Stable-Baselines3 Contributors, 2026]. La elección de PPO debe entenderse, por tanto, como una decisión de diseño para construir un pipeline experimental estable, no como evidencia previa de que la política aprendida supere a un controlador clásico.

Desde una perspectiva robótica, PPO también impone una restricción metodológica: la calidad del aprendizaje depende de la definición del entorno. Observaciones mal normalizadas, acciones con escalas inadecuadas, recompensas ambiguas o reinicios poco fiables pueden producir políticas aparentemente entrenadas pero incapaces de resolver la tarea. Por ello, los capítulos posteriores no se limitan a indicar que se usa PPO, sino que describen explícitamente el contrato Gymnasium, la formulación de observaciones y acciones, la recompensa, las condiciones de finalización y la infraestructura de validación.

2.3. Trabajo relacionado: Simuladores y entornos de entrenamiento para RL en drones

El entrenamiento mediante aprendizaje por refuerzo en robótica aérea depende fuertemente del simulador empleado. Un simulador adecuado debe ofrecer fidelidad dinámica suficiente para capturar el comportamiento del vehículo, pero también un coste

2.3. Trabajo relacionado: Simuladores y entornos de entrenamiento para RL en drones

computacional reducido que permita generar experiencia de entrenamiento [Chan et al., 2024]. La literatura reciente coincide en que este equilibrio entre realismo, velocidad y facilidad de integración es uno de los factores determinantes del éxito de una solución basada en aprendizaje por refuerzo [Azar et al., 2021; Chan et al., 2024].

Entre los entornos influyentes se encuentra Flightmare, un simulador de cuadricópteros diseñado para desacoplar el renderizado y la física, lo que permite combinar flexibilidad con una velocidad de simulación elevada [Song et al., 2021]. Su interés para aprendizaje por refuerzo reside en que puede simular centenares de drones en paralelo, lo cual lo convierte en una referencia cuando el objetivo es maximizar el rendimiento de muestreo. Desde otra perspectiva, Air Learning amplió el uso de AirSim con un entorno tipo Gym para navegación autónoma, introduciendo aleatorización del dominio, *hardware-in-the-loop* y métricas de “calidad de vuelo” orientadas al despliegue en plataformas con recursos limitados [Krishnan et al., 2021]. Este trabajo muestra que el valor de un simulador para aprendizaje por refuerzo no depende solo de la dinámica, sino también de su capacidad para aproximar las restricciones computacionales del sistema final.

En una dirección más ligera y abierta, Panerati et al. propusieron un entorno Gym basado en PyBullet para cuadricópteros individuales y multiagente, explícitamente orientado a facilitar la comparación entre enfoques de control clásico y de aprendizaje [Panerati et al., 2021]. Su aportación es especialmente interesante para trabajos académicos porque reduce la fricción de entrada, ofrece portabilidad y proporciona ejemplos tanto de control PID como de tareas de aprendizaje por refuerzo. Finalmente, *Learning to Fly in Seconds* plantea un simulador extremadamente optimizado para entrenamiento de controladores de bajo nivel, subrayando que la eficiencia computacional puede llegar a ser tan importante como la fidelidad dinámica cuando el objetivo es acelerar el ciclo de investigación [Eschmann et al., 2024].

Las revisiones de conjunto confirman esta diversidad de soluciones. Chan et al. comparan simuladores orientados a aprendizaje por refuerzo desde la perspectiva de sus prestaciones, interfaces y capacidades de integración, y concluyen que no existe una herramienta universalmente superior, sino distintas elecciones dependiendo del problema, del tipo de aprendizaje y del nivel de realismo requerido [Chan et al., 2024]. Desde la perspectiva de este trabajo, esta conclusión es importante: el simulador de multirrotores de Aerostack2 puede no ofrecer el fotorrealismo de otras plataformas, pero su ejecución sin interfaz gráfica y su integración natural con ROS 2 lo hacen atractivo para construir una infraestructura de entrenamiento conectada con un *stack* robótico real.

En este punto conviene precisar el alcance de la vectorización. El soporte para varios drones en un único simulador compartido y un único proceso Python permite validar la viabilidad funcional de escenarios con N instancias del entorno. En la infraestructura de entrenamiento preparada, la opción prudente es utilizar vectorización síncrona, por ejemplo mediante `DummyVecEnv`, antes de plantear esquemas asíncronos o afirmaciones de aceleración. Sin métricas de rendimiento comparables, la vectorización debe presentarse como capacidad de la infraestructura, no como mejora cuantificada del entrenamiento.

2.4. Síntesis y posicionamiento del trabajo

La revisión anterior permite extraer varias conclusiones. En primer lugar, el aprendizaje por refuerzo ha demostrado ser una herramienta válida en la literatura para distintos problemas de control de drones, incluyendo estabilización, seguimiento, navegación y vuelo agresivo [Kaufmann et al., 2020; Pi et al., 2021; Yoo et al., 2021; Song et al., 2023]. En segundo lugar, el rendimiento final depende de forma decisiva del nivel de abstracción de la acción, del diseño de la recompensa y de la calidad del simulador empleado [Kaufmann et al., 2022; Chan et al., 2024]. En tercer lugar, muchos trabajos priorizan simuladores especializados o entornos Gym independientes, mientras que la integración reproducible con un ecosistema robótico basado en ROS 2 introduce problemas de ingeniería que también son relevantes para la investigación aplicada [Fernández-Cortizas et al., 2023; Song et al., 2021; Krishnan et al., 2021; Panerati et al., 2021].

A partir de las fuentes revisadas, puede inferirse una oportunidad de trabajo en la construcción de entornos de aprendizaje por refuerzo vectorizables y directamente integrados con un *stack* robótico operativo como Aerostack2. Esta necesidad es más clara cuando el problema de control no se formula a nivel de actuadores, sino a un nivel intermedio próximo a la ejecución de misiones, es decir, recibir una referencia relativa al objetivo y generar comandos de velocidad compatibles con la arquitectura del dron, incorporando además una restricción de *path-facing* para alinear el yaw con la dirección de movimiento.

En este punto se sitúa el presente Trabajo Fin de Máster. La contribución se articula en dos planos. El primero es una integración reproducible entre Gymnasium, Stable-Baselines3, ROS 2 y Aerostack2: el entorno `AS2TestEnv-v0`, las configuraciones experimentales, la supervisión de campañas largas y la validación de la vectorización. El segundo es la resolución de la tarea de navegación propuesta mediante un curriculum de distancia, con una evaluación final sobre episodios reproducibles que incluye la comparación directa con un controlador PID de referencia sobre los mismos episodios.

Desde esta perspectiva, el trabajo se alinea con la literatura reciente sobre control aprendido para cuadricópteros, pero mantiene el rigor de la infraestructura experimental como condición previa de cualquier afirmación de rendimiento. Esta decisión es importante porque, sin una interfaz estable, una formulación clara de observaciones y acciones, una recompensa trazable y una estrategia de evaluación reproducible, la comparación entre aprendizaje por refuerzo y control clásico carecería de una base experimental sólida.

Capítulo 3

Diseño del sistema

Este capítulo presenta el diseño del sistema desarrollado para formular y ejecutar un entorno de aprendizaje por refuerzo orientado al control de un UAV en Aerostack2. La descripción se centra en la arquitectura, en la abstracción del problema de control y en las decisiones que permiten integrar Gymnasium con la interfaz robótica de ROS 2. No se aborda aquí la programación detallada de los módulos, sino las decisiones de diseño que estructuran la solución y delimitan su comportamiento.

3.1. Arquitectura general

La arquitectura se organiza como una cadena de responsabilidades que separa el entrenamiento del agente, la definición del entorno y la comunicación con el simulador. En el nivel superior se sitúan los scripts de entrenamiento, encargados de instanciar el entorno y conectar el algoritmo de aprendizaje por refuerzo con la API estándar de Gymnasium. Esta API actúa como frontera entre el agente y el sistema robótico, de modo que el agente no opera directamente sobre ROS 2, sino sobre un contrato de observaciones, acciones, recompensas y condiciones de finalización.

El componente central de esta frontera es la clase `AS2TestEnv`, definida como una especialización de `gym.Env` y registrada como `AS2TestEnv-v0`. Su función de diseño consiste en traducir cada paso del entorno en órdenes comprensibles por Aerostack2 y, a la inversa, convertir el estado relativo del UAV en una observación apta para el agente. La comunicación con Aerostack2 se realiza mediante `as2_python_api.DroneInterface`, mientras que el envío de velocidades se encapsula a través de `SpeedMotion` y del método `send_speed_command_with_yaw_speed`. De este modo, el agente queda desacoplado de los detalles de bajo nivel de ROS 2 y del simulador multirrotor de Aerostack2 [Fernández-Cortizas et al., 2023].

La Figura 3.1 resume esta separación por capas. La interacción avanza desde los scripts de entrenamiento hacia la API Gymnasium, el entorno `AS2TestEnv`, la interfaz `DroneInterface/SpeedMotion`, ROS 2 y, finalmente, el AS2 Multirrotor Simulator. La última capa incorpora, en la versión final del sistema, un overlay con un servicio explícito de reinicio de estado. Esta estructura sigue un criterio de aislamiento. El entrenamiento decide acciones en un espacio matemático acotado, el entorno aplica las transformaciones necesarias y Aerostack2 ejecuta los comandos en el simulador. Además, la inicialización de ROS 2 se diseña como un recurso único por proceso

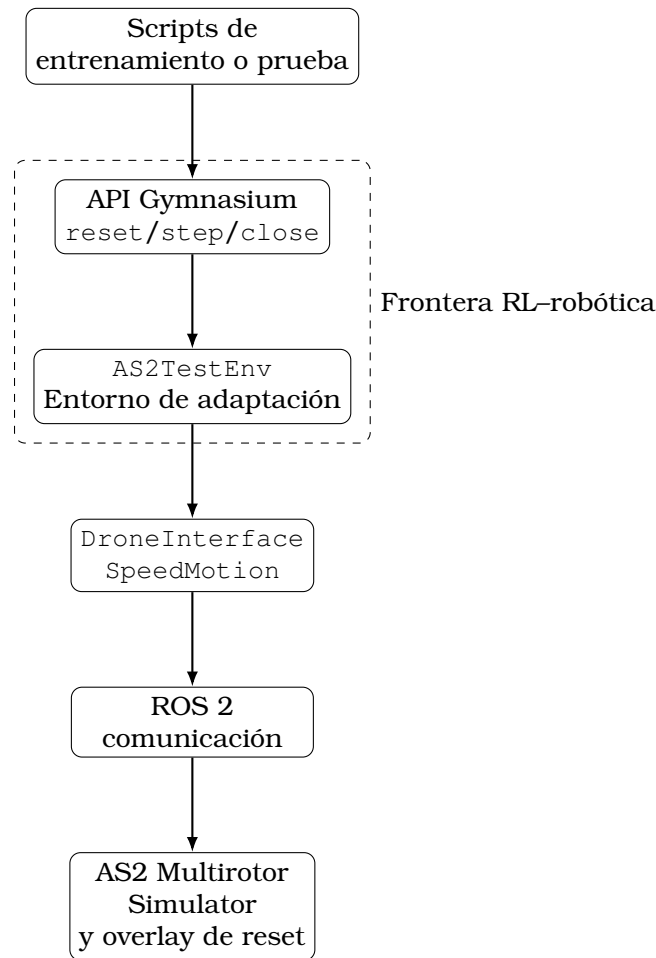


Figura 3.1: Arquitectura general del sistema, separando la lógica de aprendizaje, la interfaz Gymnasium, la comunicación robótica con Aerostack2 y el simulador con servicio de reinicio.

mediante las variables de control `_rcldpy_initialized` y `_rcldpy_lock`, con el fin de evitar inicializaciones repetidas y reducir riesgos de carrera cuando se crean varias instancias del entorno.

La incorporación del servicio de reinicio supone una parte crítica del diseño. El reinicio deja de depender únicamente de comandos de velocidad o de comportamientos de navegación de Aerostack2, y pasa a contar con una operación propia del simulador para restaurar pose, velocidades, referencias internas, controlador, odometría e IMU. Esta decisión se introduce porque, en aprendizaje por refuerzo, un episodio solo es válido si el estado inicial y la accionabilidad posterior al reset son verificables. El entorno decide cuándo reiniciar, el simulador aplica la mutación física del estado y el propio entorno vuelve a certificar telemetría y movimiento antes de entregar la siguiente transición a PPO.

3.2. Formulación del problema de control

El problema de control se formula como una tarea de navegación local hacia un objetivo relativo. En este contexto, navegación local significa que el agente no reci-

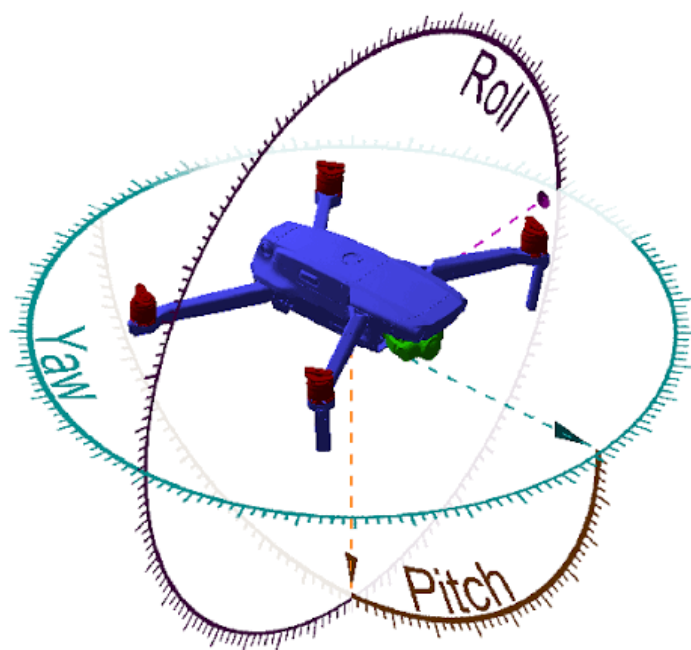


Figura 3.2: Representación conceptual de la pose de un dron mediante los ángulos de *roll*, *pitch* y *yaw*. Fuente: ResearchGate.

be una misión global completa ni una trayectoria predefinida con todos sus puntos intermedios, sino la relación geométrica entre el estado actual del UAV y un objetivo inmediato. A partir de esa relación, el agente debe emitir comandos de velocidad que acerquen el vehículo al objetivo y mantengan una orientación coherente con la dirección de desplazamiento.

Antes de definir formalmente las observaciones y acciones, conviene distinguir las magnitudes básicas que describen el movimiento de un multirrotor. La posición indica dónde se encuentra el UAV en el espacio; la velocidad indica cómo cambia esa posición; y la orientación describe la rotación del vehículo respecto a sus ejes. En un dron se suelen emplear tres ángulos principales, denominados *roll*, *pitch* y *yaw*. El *roll* representa la inclinación lateral, el *pitch* la inclinación hacia delante o hacia atrás, y el *yaw* la rotación alrededor del eje vertical. La Figura 3.2 resume esta idea de pose del dron mediante una representación conceptual tomada de ResearchGate.¹

En el presente trabajo no se controla directamente la actitud completa del dron ni las velocidades de giro de cada rotor. La acción se define a un nivel de abstracción superior, en el que el agente genera velocidades lineales y una velocidad de guiñada. Esta elección sitúa el aprendizaje por refuerzo por encima del control de bajo nivel y delega en Aerostack2 la ejecución de los comandos sobre el simulador. Desde el punto de vista del diseño, esta separación reduce la complejidad del problema aprendido y mantiene la compatibilidad con la interfaz de control de velocidad disponible.

La formulación distingue también entre la dirección frontal del UAV y la dirección real de avance. Un dron puede desplazarse hacia un objetivo aunque su eje frontal

¹Imagen disponible en ResearchGate: https://www.researchgate.net/figure/A-conceptual-1-representation-of-drone-pose-A-drone-here-represented-by-a-DJI-Mavic-2-is_fig1_356985671. Según la página consultada, la figura aparece con licencia Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International.

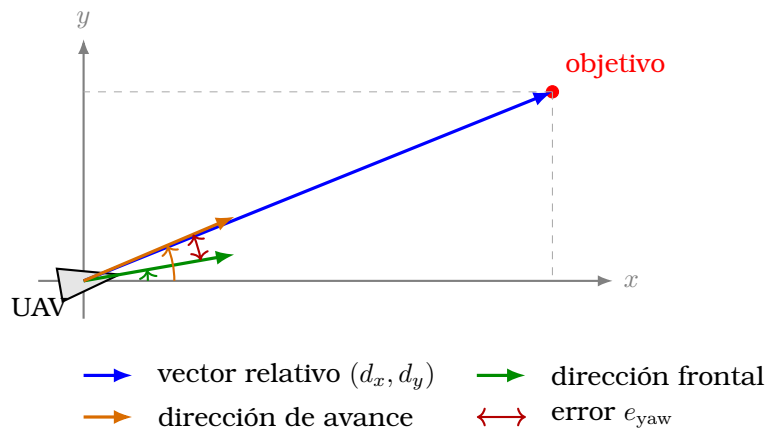


Figura 3.3: Relación geométrica entre el UAV, el objetivo relativo, la dirección de avance y la orientación de guiñada utilizada en el criterio de *path-facing*.

no esté alineado con la trayectoria seguida. Por ello, el diseño inicial introduce el criterio de *path-facing*, con el que se buscaba que la guiñada fuese coherente con la dirección horizontal del movimiento además de reducir la distancia al objetivo. La Figura 3.3 ilustra esta relación entre posición actual, objetivo, vector relativo, dirección de avance y error de orientación.

La acción se define mediante el vector continuo

$$a = [v_x, v_y, v_z, v_{yaw}].$$

Las tres primeras componentes representan velocidades lineales en los ejes cartesianos y la cuarta representa la velocidad angular de guiñada. El espacio de acciones se acota mediante los parámetros `max_vel` y `max_yaw_vel`, lo que introduce una restricción explícita de diseño sobre la magnitud de las órdenes que el agente puede emitir. Esta acotación es necesaria porque una política de aprendizaje por refuerzo puede proponer acciones arbitrarias dentro de su espacio de salida, mientras que el sistema físico o simulado solo debe recibir comandos dentro de un rango admisible.

La observación se define mediante el vector relativo

$$o = [d_x, d_y, d_z, d_{yaw}].$$

Cada componente se normaliza en el intervalo $[-1, 1]$. Las tres primeras componentes codifican el desplazamiento relativo hacia el objetivo, y la cuarta codifica la diferencia relativa de guiñada. La normalización reduce la dependencia respecto a la escala concreta del escenario y proporciona al agente entradas con rangos homogéneos, una condición práctica para estabilizar el tratamiento de las observaciones por parte del algoritmo de aprendizaje.

3.3. Diseño del entorno Gymnasium

El entorno Gymnasium se diseña como una adaptación entre el ciclo estándar `reset-step-close` y las operaciones necesarias para preparar, gobernar y finalizar el vuelo simulado. Esta decisión permite que el agente interactúe con el UAV mediante una

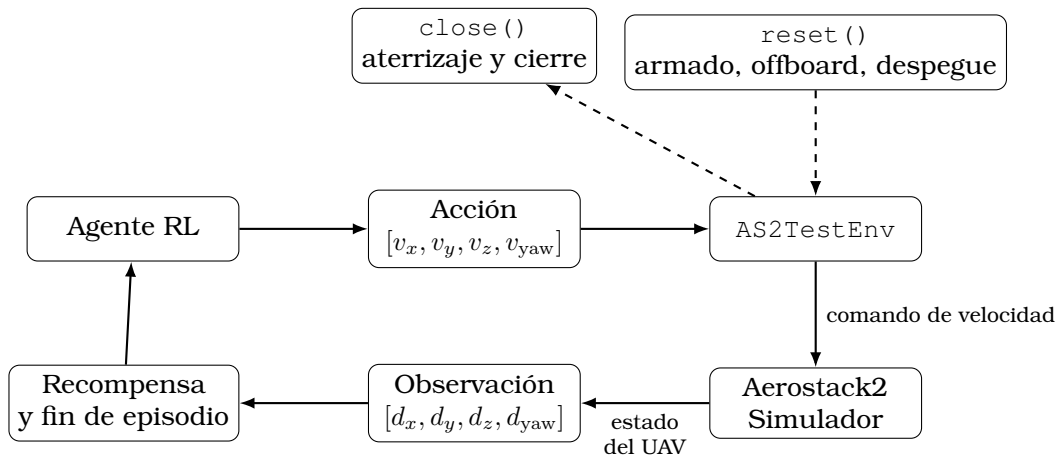


Figura 3.4: Ciclo de interacción entre el agente, el entorno Gymnasium y el simulador durante un episodio.

interfaz conocida en aprendizaje por refuerzo, mientras que la lógica específica de Aerostack2 permanece encapsulada dentro del entorno.

El método `reset` establece el estado inicial de un episodio. Su diseño incluye el armado del UAV, la activación del modo *offboard*, el despegue y la devolución de la primera observación junto con la información auxiliar del episodio. Por tanto, el reinicio no se limita a reinicializar variables internas, sino que prepara al sistema simulado para aceptar comandos de velocidad desde el agente. Esta preparación es relevante porque el contrato de Gymnasium presupone que, tras `reset`, el entorno está listo para ejecutar `step` sin operaciones adicionales por parte del algoritmo.

El método `step` constituye el núcleo de la interacción. En cada llamada, el entorno recibe la acción continua del agente, la transforma en una orden de velocidad enviada mediante `SpeedMotion`, actualiza la observación relativa y calcula la recompensa y los indicadores de fin de episodio. El resultado sigue el contrato de Gymnasium y devuelve observación, recompensa, indicador de terminación, indicador de truncamiento e información auxiliar. La separación entre *terminación* y *truncamiento* permite distinguir un episodio que acaba por alcanzar una condición del problema de otro que finaliza por límite externo de pasos.

El método `close` se diseña como una operación segura de cierre del entorno. Su responsabilidad es aterrizar el UAV y liberar o cerrar los recursos asociados a la comunicación con Aerostack2. Esta decisión busca reducir el riesgo de que el final de un experimento deje el simulador en un estado inconsistente y concentra la lógica de apagado en el mismo componente que gestiona la interacción durante el episodio.

El entorno contempla también un modo de diseño denominado `randomize_hover_start`. Cuando está activo, el inicio, el objetivo y la guiñada del episodio se muestrean dentro de límites configurables y se evitan casos triviales mediante una separación mínima entre inicio y objetivo. Las configuraciones más recientes incorporan además un margen de muestreo respecto a las fronteras de la escena y un límite máximo de distancia inicio-objetivo. Esta decisión evita que las observaciones relativas se saturen en $[-1, 1]$ desde el inicio del episodio y reduce la probabilidad de que un agente no entrenado genere inmediatamente una terminación fuera de límites.

La Figura 3.4 muestra el ciclo de interacción que se repite durante un episodio. A partir de este ciclo, la Cuadro 3.1 sintetiza el contrato principal que el entorno ofrece al algoritmo de aprendizaje.

Elemento	Diseño adoptado
Observación	Vector relativo normalizado $[d_x, d_y, d_z, d_{yaw}]$
Acción	Velocidades continuas $[v_x, v_y, v_z, v_{yaw}]$
Recompensa	Distancia normalizada al objetivo y bonificación terminal
Finalización	Éxito, salida de límites, baja altitud insegura o truncamiento temporal

Cuadro 3.1: Resumen del contrato principal del entorno Gymnasium.

3.4. Diseño del reinicio de episodios y accionabilidad

El reinicio de episodios constituye una decisión de diseño central en un entorno de aprendizaje por refuerzo para robótica aérea. En un entorno puramente numérico, reiniciar un episodio puede consistir en asignar directamente un nuevo estado inicial. En un simulador robótico integrado con ROS 2 y Aerostack2, en cambio, el reinicio afecta al estado físico del multirrotor, al controlador, a los modos de operación de la plataforma y a la coherencia temporal de las lecturas publicadas. Por este motivo, el diseño del entorno no trata el método `reset` como una operación auxiliar, sino como una parte crítica de la validez experimental.

El diseño contempla varios niveles de reinicio. El nivel básico corresponde al reinicio clásico mediante armado, activación del modo *offboard* y despegue. Este procedimiento es suficiente para una primera inicialización, pero resulta lento y no siempre adecuado entre episodios de entrenamiento. Para experimentos controlados se introduce una pose inicial fija, o `fixed_start_pose`, que separa explícitamente el punto de arranque del objetivo. Cuando el UAV ya se encuentra en vuelo, el retorno a dicha pose se plantea preferentemente mediante comandos de velocidad acotados. Esta decisión evita depender de comportamientos bloqueantes y mantiene la interacción dentro del mismo tipo de órdenes que usa el agente.

También se contempla el reinicio mediante un servicio del simulador. Este mecanismo permite restablecer el estado del multirrotor de forma más directa, pero se considera una herramienta de recuperación controlada y no una solución ciega. Teletransportar un sistema dinámico de un estado terminal a un estado inicial puede generar inconsistencias si los estados internos de la dinámica o del controlador no quedan sincronizados. Por ello, el servicio de reinicio se acompaña de comprobaciones posteriores de pose, modo de operación y capacidad de aceptar comandos físicos antes de declarar válido el inicio de un nuevo episodio.

El servicio de reinicio se diseña como una operación propiedad del simulador y se invoca bajo el espacio de nombres del vehículo, por ejemplo `/drone0/platform/reset_simulator_s`. Su contrato recibe una pose objetivo y tolerancias de validación, y debe devolver evidencia inmediata de la mutación aplicada mediante errores de posición y guiñada, normas residuales de velocidad, aceleración, fuerza y par, y estado de motores. Además, sincroniza la capa de plataforma para que Aerostack2 quede conectado, armado, en modo *offboard* y en estado de vuelo. La respuesta del servicio no se considera por sí sola suficiente para iniciar el siguiente episodio; después del servicio

3.5. Diseño de la recompensa y condiciones de finalización

se comprueba la telemetría publicada y se ejecuta una prueba de accionabilidad con movimiento físico medible.

Una decisión relevante del diseño es que la aceptación de un comando por la API robótica no se considera evidencia suficiente de que la acción sea efectiva. El entorno debe distinguir entre un comando que ha sido aceptado por la interfaz y un comando que produce movimiento físico medible en el simulador. Esta diferencia es crítica para aprendizaje por refuerzo. Si el agente recibe observaciones y recompensas mientras sus acciones no modifican realmente el estado del UAV, los datos recogidos dejan de representar la tarea que se pretende aprender. Por ello, los fallos de accionabilidad se tratan como condiciones de rechazo temprano del entrenamiento y no como episodios válidos.

Estado o fallo	Significado de diseño	Respuesta prevista
reset timeout	El UAV no alcanza la pose inicial dentro del tiempo definido.	Fallo acotado del reinicio y rechazo de continuar el entrenamiento.
post_controller_reach	Tras reiniciar o estabilizar el controlador, no se recupera la tolerancia de posición.	Diagnóstico específico del acoplamiento entre reset y controlador.
post_controller_hold	La pose se alcanza de forma transitoria, pero no se mantiene estable.	Rechazo del reset para evitar datos de episodio físicamente inválidos.
Comando aceptado sin movimiento	La interfaz acepta la orden, pero no se observa desplazamiento físico suficiente.	Fallo cerrado de accionabilidad antes de iniciar o continuar el entrenamiento.
Servicio de reset aplicado sin accionabilidad	El simulador informa que la pose se ha restaurado, pero el camino de comandos posterior no produce desplazamiento suficiente.	Reintento completo del servicio y, si persiste, parada explícita del entrenamiento.

Cuadro 3.2: Estados y fallos considerados en el diseño del reinicio de episodios.

El Cuadro 3.2 resume los principales estados de fallo que guían el diseño. En todos los casos se favorece un fallo explícito y trazable frente a un bloqueo indefinido o a la continuación de un entrenamiento con datos contaminados por un estado físico no válido.

3.5. Diseño de la recompensa y condiciones de finalización

La recompensa final usada en la fase de experimentación principal se diseña para mantener una señal mínima e interpretable. En cada paso se penaliza únicamente la distancia normalizada al objetivo. Para ello, si d_t representa la distancia euclídea entre el UAV y el objetivo, y $d_{\text{máx}}$ representa la distancia máxima de normalización. A partir de ambas magnitudes se obtiene

$$d_{\text{norm}} = \min\left(\frac{d_t}{d_{\text{máx}}}, 1\right), \quad r_{\text{dist}} = -d_{\text{norm}}. \quad (3.1)$$

En el entorno implementado, $d_{\text{máx}}$ se calcula a partir del límite espacial de posición del escenario, de forma que la distancia normalizada queda acotada en el intervalo

$[0, 1]$.

El entorno conserva la capacidad de calcular términos adicionales, como progreso, seguridad vertical o coherencia de guiñada con la dirección de movimiento. Sin embargo, esos términos se desactivan a partir de Exp010 y en toda la campaña principal para evitar que el agente optimice una recompensa de shaping distinta de la tarea principal. La guiñada se mantiene como requisito terminal suave. Cuando el UAV alcanza la región objetivo, la bonificación de éxito se reduce en función del error de guiñada. Si e_{yaw} representa el error angular normalizado, la recompensa terminal de éxito se calcula mediante

$$r_{\text{success}} = R_{\text{success}} - w_{\text{yaw}} \frac{|e_{\text{yaw}}|}{\pi}. \quad (3.2)$$

La expresión final aplicada desde Exp010 en adelante incorpora las condiciones terminales del episodio de la forma siguiente.

$$r_t = \begin{cases} -P_{\text{oob}}, & \text{si el UAV sale de los límites del escenario,} \\ -P_{\text{oob}}, & \text{si el UAV entra en baja altitud insegura,} \\ r_{\text{success}}, & \text{si } d_t < d_{\text{threshold}}, \\ -d_{\text{norm}}, & \text{en el resto de pasos.} \end{cases} \quad (3.3)$$

El truncamiento por alcanzar `max_steps` no introduce una recompensa adicional, sino que limita la duración del episodio cuando no se ha alcanzado una condición terminal. El Cuadro 3.3 resume los componentes de esta formulación y su efecto esperado.

3.6. Guards de acción y protección de fronteras

La observación relativa tiene una consecuencia de diseño que los primeros experimentos hicieron explícita: el agente no puede percibir los límites del escenario. Una terminación por salida de límites es, desde el punto de vista de la política, un castigo imprevisible aplicado en estados que no se distinguen de los estados seguros, por lo que la función de valor no puede anticiparla. Además, cada una de estas terminaciones obliga a un reinicio mediante el servicio del simulador, con un coste de decenas de segundos y efectos acumulativos sobre la plataforma. Por ambos motivos, el diseño sustituye la muerte en frontera por una protección activa: un *guard* de acción que, cerca de los límites, reemplaza la componente de velocidad saliente por un empuje hacia el interior.

El guard se diseña con disparo predictivo. La versión reactiva, que actúa cuando la posición cruza una línea de guardia, resultó insuficiente en el eje vertical ya que la plataforma invierte la velocidad con lentitud y la inercia atraviesa el margen antes de que la orden correctora surta efecto. Por ello el disparo evalúa la posición *predicha*, calculada como posición actual más velocidad medida por un horizonte de anticipación, y el empuje vertical utiliza la autoridad completa de velocidad disponible. Las terminaciones por salida de límites y por baja altitud se conservan intactas como red de seguridad final, pero con el guard activo pasan de ser el desenlace dominante (80–93 % de los episodios en Exp010–Exp011) a ser residuales, lo que elimina a la vez el ruido inaprendible de la recompensa y la mayor parte de los reinicios costosos.

3.7. Diseño del curriculum de distancia

Componente	Expresión	Condición	Efecto esperado
Penalización por distancia	$r_{\text{dist}} = -d_{\text{norm}}$	En cada paso	Favorece que el UAV reduzca progresivamente la distancia al objetivo.
Bonificación por éxito	$R_{\text{success}} - w_{\text{yaw}} e_{\text{yaw}} /\pi$	Si $d_t < d_{\text{threshold}}$	Premia alcanzar la región objetivo y reduce la bonificación si la guiñada terminal es inadecuada.
Penalización por salida de límites	$-P_{\text{oob}}$	Si el UAV supera los límites del escenario	Desincentiva trayectorias inseguras o alejadas de la zona de entrenamiento.
Penalización por baja altitud insegura	$-P_{\text{oob}}$	Si el UAV desciende por debajo del umbral inseguro	Evita que el aprendizaje explote estados cercanos al suelo o físicamente no válidos.
Truncado temporal	Sin recompensa adicional	Si se alcanza el número máximo de pasos	Limita la duración del episodio cuando no se alcanza una condición terminal.

Cuadro 3.3: Componentes de la función de recompensa y condiciones de finalización del entorno.

3.7. Diseño del curriculum de distancia

Con la recompensa mínima $-d_{\text{norm}}$, la única señal positiva del problema es la bonificación terminal, y su densidad depende de la probabilidad de que la exploración alcance la esfera de éxito de 0,4 m. Con separaciones inicio-objetivo de hasta 5 m esa probabilidad es demasiado baja para arrancar el gradiente con la tasa de aprendizaje configurada. El diseño adopta por ello un curriculum de distancia [Bengio et al., 2009] en tres etapas: la primera acota la separación máxima a 2,5 m, de modo que un vuelo directo requiere del orden de 25 pasos; las dos siguientes amplían el límite a 3,5 m y a los 5,0 m de la tarea original. Cada etapa se inicializa con el modelo final de la anterior (*warm start*) y todas mantienen la separación mínima de 1,0 m, de forma que la distribución de episodios de una etapa contiene a la anterior y la política no olvida los objetivos cercanos al aprender los lejanos.

La Figura 3.5 resume esta organización mediante varias instancias lógicas conectadas a una infraestructura de simulación compartida.

3.8. Diseño de la ejecución vectorizada

La ejecución vectorizada se diseña para permitir varias instancias lógicas del entorno sobre un único simulador compartido. El diseño utiliza espacios de nombres diferenciados, `drone0` a `droneN`, de forma que cada UAV simulado pueda ser identificado y gobernado de manera independiente dentro de la misma infraestructura ROS 2. Esta organización reduce la necesidad de duplicar toda la infraestructura del simulador,

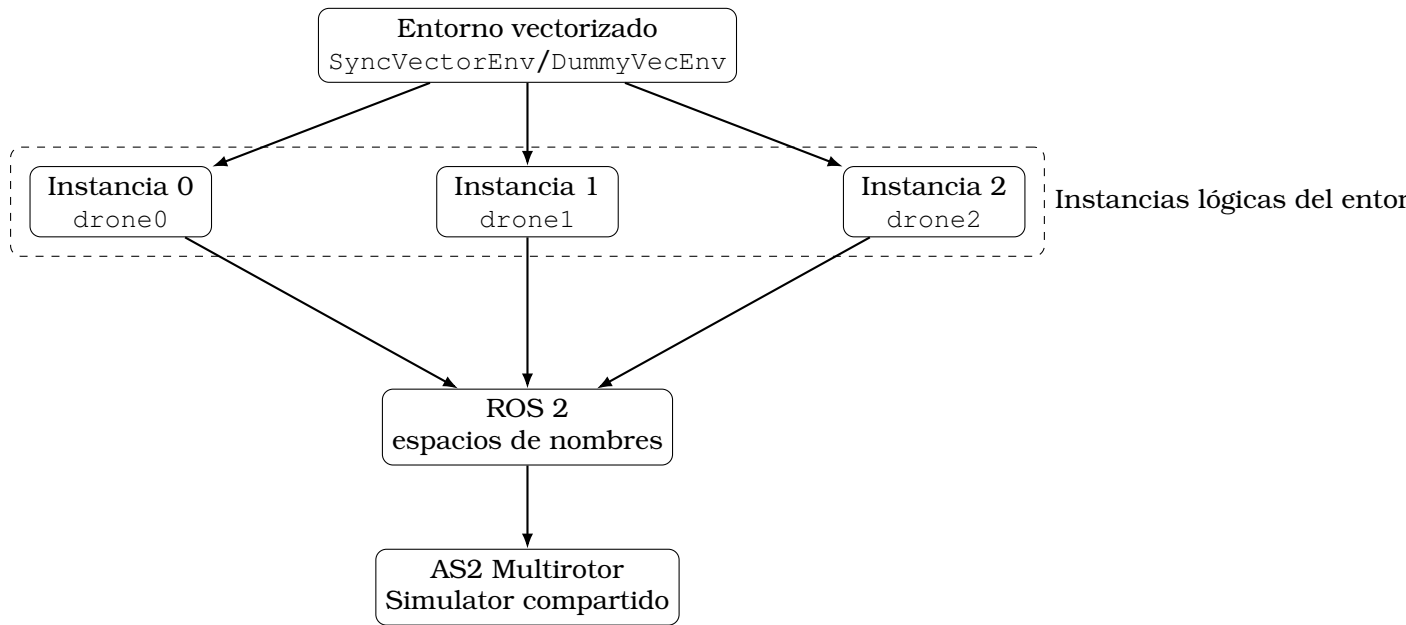


Figura 3.5: Diseño de la ejecución vectorizada mediante varias instancias lógicas asociadas a espacios de nombres diferenciados sobre una infraestructura de simulación compartida.

pero exige tratar con cuidado la inicialización de ROS 2, los nombres de los drones y la sincronización entre episodios.

La elección del mecanismo de vectorización viene determinada por una propiedad del simulador: opera en tiempo de reloj real, de modo que cada paso del entorno consume la duración física configurada (0,2 s) con independencia del proceso que lo invoque. Esta propiedad invalida en este sistema el razonamiento habitual a favor de la vectorización secuencial (`SyncVectorEnv` en `Gymnasium`, `DummyVecEnv` en `Stable-Baselines3`), cuyo beneficio procede de agrupar las observaciones para una única inferencia por lotes: con un simulador de reloj real, N entornos secuenciales tardan N veces el paso físico en producir N muestras, es decir, el mismo caudal de muestreo que un solo entorno. Por ello el diseño adopta `SubprocVecEnv`, que ejecuta los pasos de los N entornos en paralelo real manteniendo la inferencia por lotes, con la expectativa (verificada experimentalmente) de multiplicar el muestreo por el número de drones.

El paso vectorizado en paralelo introduce, sin embargo, un modo de fallo propio de los simuladores de reloj real, el avance en *lockstep*. Cuando uno de los entornos ejecuta su reinicio, es una operación que tarda decenas de segundos, el paso vectorizado queda bloqueado para todos; durante ese intervalo los demás drones no reciben órdenes nuevas mientras el mundo sigue avanzando, y el controlador de `Aerostack2` mantiene la última referencia de velocidad como consigna activa. El diseño incorpora por ello un mecanismo de vigilancia (*watchdog*) de comandos inactivos: si un entorno en vuelo supera un umbral de tiempo sin emitir referencias de movimiento, el mecanismo publica una consigna de velocidad nula de forma periódica hasta que el flujo normal de comandos se reanuda. El umbral se dimensiona por encima de la duración del paso, de modo que el mecanismo permanece inerte durante la operación normal

y solo actúa en los huecos de *lockstep*; en ejecución monodron queda desactivado y el comportamiento del entorno es idéntico al de la fase anterior.

3.9. Diseño del protocolo de evaluación

La comparación entre la política aprendida y un controlador clásico exige que ambos se evalúen exactamente sobre los mismos episodios. El protocolo se diseña en torno a dos decisiones. La primera es la reproducibilidad por semillas: el episodio i de una evaluación se genera reiniciando el entorno con la semilla $s_0 + i$, de modo que la secuencia de pares inicio–objetivo es idéntica entre evaluaciones y controladores. La segunda es la abstracción del controlador: el protocolo define una interfaz mínima (reiniciar estado interno al comienzo del episodio y transformar una observación en una acción) que cumplen tanto la política PPO, en modo determinista o estocástico, como un controlador PID por ejes que consume la misma observación relativa normalizada y emite acciones en el mismo espacio acotado. Cada evaluación registra métricas por episodio (éxito, razón de terminación, pasos, distancia final, longitud de trayectoria, error de guiñada terminal y normas de acción) y las agrega globalmente y por tramos de distancia inicial, lo que permite comparar controladores con evidencia cuantitativa homogénea.

Capítulo 4

Implementación

Este capítulo describe cómo se materializa en el repositorio el diseño definido en el capítulo anterior. La atención se desplaza desde la formulación del problema hacia la organización del software, la integración efectiva con ROS 2 y Aerostack2, la configuración de PPO y los mecanismos de diagnóstico necesarios para ejecutar experimentos reproducibles.

La implementación no constituye un controlador de referencia clásico. El sistema construye una infraestructura de entrenamiento en la que Aerostack2 conserva el control de bajo nivel, el armado, el despegue y la ejecución simulada, mientras que la política aprendida actúa mediante comandos de velocidad compatibles con la plataforma. Esta distinción evita atribuir a la política responsabilidades que, en la solución desarrollada, pertenecen al *stack* robótico.

4.1. Organización del software

El código desarrollado se distribuye en dos repositorios públicos. El repositorio principal `rl_uav_aerostack2`¹ contiene el entorno de aprendizaje, las configuraciones de entrenamiento, los scripts de ejecución y las utilidades de validación. El repositorio `as2_platform_multirotor_simulator`² contiene el fork del simulador empleado para añadir el servicio de reinicio determinista. Esta separación hace explícito qué parte corresponde al entorno Gymnasium y qué parte modifica la capa física de simulación.

La implementación se organiza alrededor de tres puntos de entrada. El módulo `rl_uav/envs/as2_test_env.py` contiene la clase `AS2TestEnv` y concentra la traducción entre Gymnasium y Aerostack2. El fichero `configs/train_ppo.yaml` define de forma declarativa los parámetros del entorno, la vectorización, las rutas de salida y los hiperparámetros de PPO. Por último, `scripts/train_ppo.py` lanza el entrenamiento o valida la configuración mediante un modo *dry-run*. Esta división permite cambiar una configuración experimental sin modificar la lógica del entorno ni los scripts de ejecución.

El repositorio incluye además utilidades de validación para comprobar la conexión

¹https://github.com/KafeisM/rl_uav_aerostack2

²https://github.com/KafeisM/as2_platform_multirotor_simulator

4.2. Integración con Aerostack2 y ROS 2

monodron, la ejecución vectorizada, el inicio aleatorizado en hover y el comportamiento local con *mocks*. Estas comprobaciones no sustituyen a los experimentos, pero evitan confundir errores de infraestructura con errores de aprendizaje. En paralelo, el fork del simulador incorpora el servicio `ResetSimulatorState`, necesario para reiniciar de forma controlada el estado físico del multirrotor durante las campañas experimentales.

Elemento	Responsabilidad principal
<code>rl_uav/envs/as2_test_env.py</code>	Entorno Gymnasium y puente con Aerostack2.
<code>rl_uav/training/ppo_setup.py</code>	Construcción de entorno vectorizado, rutas de ejecución y modelo PPO.
<code>configs/train_ppo.yaml</code>	Configuración declarativa de entorno, entrenamiento e hiperparámetros.
<code>scripts/train_ppo.py</code>	Punto de entrada para entrenamiento y comprobación de configuración.
<code>scripts/as2_runtime.py</code>	Comprobación y relanzamiento controlado del simulador AS2.
<code>scripts/supervise_train.sh</code>	Supervisor de entrenamientos largos con reinicio del stack AS2 y reanudación desde <i>checkpoint</i> .
<code>rl_uav/evaluation/</code>	Protocolo de evaluación por semillas con controladores enchufables, PPO y PID.
<code>scripts/evaluate_policy.py</code>	Punto de entrada de la evaluación formal y de la comparación entre controladores.
<code>as2_platform_multirrotor_simulator</code>	Fork del simulador con servicio de reinicio determinista del estado del multirrotor.
<code>as2_sim/launch_sim.bash</code>	Arranque del simulador monodron o multidron mediante nombres de dron.
<code>scripts/test_* y</code> <code>scripts/validate_*</code>	Validaciones locales y reales del entorno, la vectorización y la integración.

Cuadro 4.1: Organización principal del software implementado.

El Cuadro 4.1 resume esta división. No se busca enumerar todos los ficheros del repositorio, sino mostrar que cada bloque tiene una responsabilidad acotada. El entorno define la interacción, la configuración describe el experimento, los scripts ejecutan tareas reproducibles y las utilidades de validación reducen el riesgo de confundir un fallo de simulación con un fallo del algoritmo.

4.2. Integración con Aerostack2 y ROS 2

La integración implementada toma como punto de partida la infraestructura presentada en el Capítulo 2. ROS 2 aporta el grafo de comunicación robótica y Aerostack2 proporciona la capa aérea que encapsula operaciones como armado, modo *offboard*, despegue, aterrizaje y envío de velocidades [Fernández-Cortizas et al., 2023; Open Robotics, 2026]. En este capítulo no se repite su fundamento tecnológico; se describe cómo el entorno `AS2TestEnv` utiliza esas interfaces para convertir acciones de PPO en órdenes ejecutables por el simulador.

La Figura 4.1 muestra la integración implementada. El agente PPO interactúa únicamente con la interfaz Gymnasium. La clase `AS2TestEnv` recibe las acciones, las convierte en comandos de velocidad y utiliza `DroneInterface` y `SpeedMotion` para comunicarse con Aerostack2. A su vez, Aerostack2 utiliza ROS 2 para transportar las órdenes y estados hacia el simulador multirrotor.

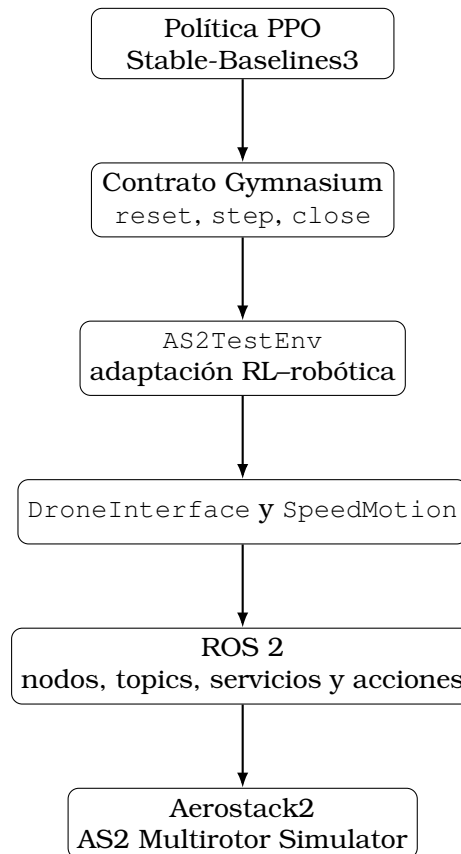


Figura 4.1: Integración implementada entre la política de aprendizaje, la interfaz Gymnasium, Aerostack2 y ROS 2.

En el código, la inicialización de ROS 2 se encapsula dentro de `AS2TestEnv`. El entorno protege la llamada a `rclpy.init()` con un bloqueo y una bandera de clase para evitar inicializaciones repetidas dentro del mismo proceso. Esta decisión es especialmente relevante en escenarios vectorizados, donde varias instancias lógicas del entorno pueden coexistir y compartir recursos de ROS 2. Tras inicializar ROS 2, el entorno crea una instancia de `DroneInterface` asociada al espacio de nombres del dron, por ejemplo `drone0`, y una instancia de `SpeedMotion` para el envío de velocidades.

El método `reset` ejecuta las operaciones necesarias para preparar un episodio. Inicializa la interfaz si aún no existe, arma el dron, activa el modo *offboard*, ejecuta el despegue y genera la primera observación. El método `step` aplica la acción emitida por el agente como un comando de velocidad lineal y velocidad de guiñada mediante `send_speed_command_with_yaw_speed`. Se emplea el marco de referencia *earth*, de forma coherente con la formulación del entorno como navegación local hacia un objetivo relativo. El método `close`, por su parte, realiza un cierre seguro mediante aterrizaje, paso a modo manual y liberación de la interfaz del dron. No se apaga globalmente `rclpy` desde cada instancia, porque hacerlo podría invalidar otras instancias activas del entorno.

El simulador se lanza desde `as2_sim/launch_sim.bash`. Este script permite seleccionar el número de drones y genera nombres de espacio como `drone0`, `drone1`, etc.

Para la ejecución multidron se adopta un criterio de un único simulador compartido con varios espacios de nombres ROS 2, en lugar de iniciar varios simuladores independientes. Esta decisión reduce la sobrecarga de infraestructura y se alinea con la idea de Aerostack2 como marco capaz de operar sistemas aéreos multirrobot [Fernández-Cortizas et al., 2023]. Para hacer más robusta la ejecución, `scripts/as2_runtime.py` comprueba la existencia de las sesiones del simulador y permite relanzarlas antes de ejecutar validaciones que dependen de AS2.

Para los experimentos con reinicio certificado se utiliza un overlay del AS2 Multirrotor Simulator. El servicio añadido se expone bajo el espacio de nombres del vehículo, por ejemplo `/drone0/platform/reset_simulator_state`, y permite aplicar una pose objetivo directamente sobre el estado del simulador. La implementación no se limita a cambiar posición y orientación, sino que también pone a cero velocidades, aceleraciones, fuerzas, pares y estado de motores, reinicia controlador, IMU y odometría, y resincroniza la plataforma de Aerostack2 para dejarla armada, en modo *offboard* y en estado de vuelo.

4.3. Configuración operativa de PPO

La configuración de PPO se materializa en `configs/train_ppo.yaml`. El Capítulo 2 justifica la elección del algoritmo; esta sección se limita a documentar cómo se parametriza su uso dentro del sistema implementado y cómo se preserva la trazabilidad de cada ejecución.

La configuración se divide en tres bloques principales. El bloque `experiment` define el nombre del experimento, la semilla y el directorio raíz de salida. El bloque `environment` agrupa los parámetros del entorno, incluyendo el identificador `AS2TestEnv-v0`, el número de instancias vectorizadas, el prefijo de espacio de nombres, los límites de posición y velocidad, la duración de cada paso, el objetivo, las condiciones de éxito y las penalizaciones de seguridad. El bloque `ppo` contiene los hiperparámetros del algoritmo, como la tasa de aprendizaje, el número de pasos por actualización, el tamaño de lote, el factor de descuento, el rango de *clipping* y la arquitectura de las redes de política y valor.

Implementación

Parámetro	Valor base	Función
total_timesteps	1000000	Duración máxima de la campaña de entrenamiento.
learning_rate	3.0e-05	Tamaño de actualización de los parámetros de PPO.
n_steps	512	Número de pasos recogidos antes de cada actualización.
batch_size	32	Tamaño de lote usado durante la optimización.
clip_range	0.2	Límite de variación entre política antigua y política nueva.
use_sde	true	Activación de exploración dependiente del estado.
net_arch	[128, 128]	Capas ocultas de las redes de política y valor.

Cuadro 4.2: Principales parámetros PPO definidos en la configuración base de entrenamiento.

El Cuadro 4.2 recoge los parámetros más representativos de la configuración base. Estos valores no deben interpretarse como una optimización definitiva del algoritmo, sino como una configuración inicial trazable y reproducible para validar la infraestructura de entrenamiento. Los capítulos de pruebas y resultados son los que deben determinar si una configuración concreta produce aprendizaje útil, estabilidad suficiente o necesidad de ajuste.

A partir de Exp010 se emplea una configuración más óptima con tasa de aprendizaje 3.0e-05, n_steps=512, batch_size=32, n_epochs=5, ent_coef=0.0, exploración SDE y redes MLP de dos capas de 128 unidades por rama. Las configuraciones Exp010, Exp011 y Exp012 mantienen estos hiperparámetros y modifican principalmente propiedades del entorno y de la tarea, como la recompensa mínima, la randomización completa, los márgenes de muestreo, el límite de distancia inicio-objetivo, los guards de acción y la reanudación de entrenamiento. Esta separación evita mezclar ajustes del algoritmo con cambios en las condiciones de interacción del UAV.

La configuración también explicita decisiones de ingeniería que afectan a la reproducibilidad. El entrenamiento se fija inicialmente en CPU, coherente con el uso de políticas MLP y con la recomendación habitual de Stable-Baselines3 para evitar sobrecostes de GPU en redes no convolucionales. Además, se separan los directorios de *logs*, monitorización y *checkpoints*, lo que permite analizar una ejecución sin depender únicamente de la salida estándar del proceso. Esta estructura facilita repetir experimentos, comparar configuraciones y aislar fallos entre simulación, entorno y algoritmo.

4.4. Herramientas de visualización y monitorización

La implementación se apoya en herramientas de visualización y monitorización que permiten distinguir entre el funcionamiento del algoritmo, el estado del simulador y la respuesta física del UAV. Esta separación es necesaria porque un entrenamiento puede fallar por una configuración de PPO inadecuada, un reinicio inválido del epi-

4.4. Herramientas de visualización y monitorización

sodio, un problema de comunicación ROS 2 o una orden aceptada por la interfaz que no produce movimiento físico.

RViz2 se utiliza como herramienta de inspección cualitativa del sistema robótico. Aunque el simulador de multirrotores de Aerostack2 no requiere interfaz gráfica, RViz2 permite observar la posición del UAV, la trayectoria reciente y los marcadores publicados para representar el objetivo. Esta visualización no se usa como métrica de aprendizaje, pero sí como apoyo para detectar comportamientos anómalos, como ausencia de movimiento, trayectorias divergentes o incoherencias entre el objetivo configurado y la escena simulada.

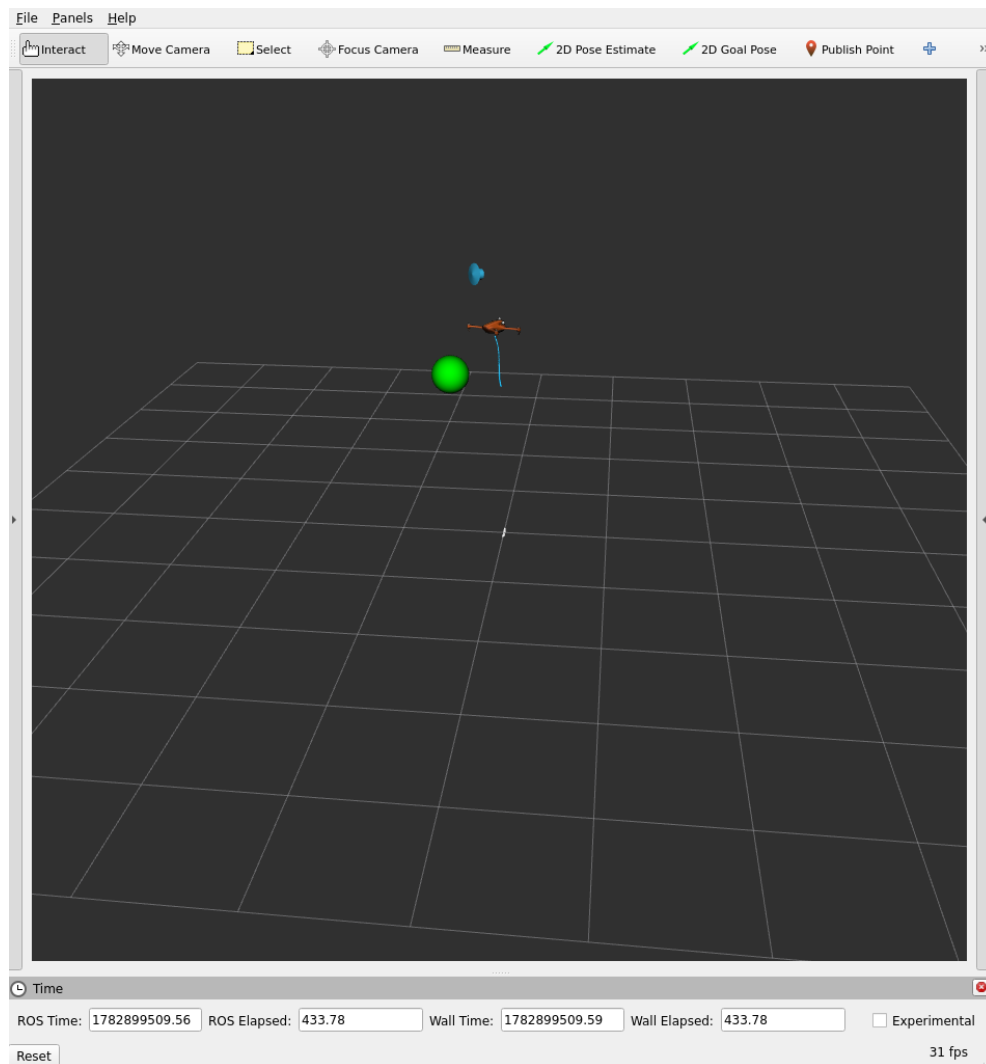


Figura 4.2: Visualización en RViz2 del UAV simulado, la trayectoria reciente y el marcador del objetivo durante una ejecución con Aerostack2.

La Figura 4.2 muestra un ejemplo de la visualización empleada durante las pruebas. El marcador verde representa el objetivo de navegación y el modelo del UAV permite comprobar de forma cualitativa si el movimiento observado es coherente con las acciones enviadas por el entorno.

TensorBoard se emplea como herramienta de monitorización de las métricas emitidas

Implementación

por Stable-Baselines3 y por los monitores vectorizados. En las ejecuciones en las que PPO alcanza suficientes pasos para completar una primera actualización, esta herramienta permite inspeccionar magnitudes como recompensa media por episodio, longitud media de episodio y otros escalares registrados. Sin embargo, la interpretación de TensorBoard debe realizarse junto con los ficheros de monitor y los diagnósticos del entorno. Una curva ausente o congelada puede indicar que el entrenamiento no ha alcanzado un *rollout* completo, que el proceso se ha bloqueado durante un reset o que el simulador ha dejado de producir transiciones válidas.

Herramienta	Uso en el trabajo
ROS 2	Comunicación entre nodos, servicios, acciones y flujos de datos del sistema robótico.
Aerostack2	Capa aérea reutilizada para armado, modos de vuelo, control y simulación.
AS2 Multirrotor Simulator	Simulación dinámica del multirrotor usado como fuente de experiencia.
Gymnasium	Interfaz estándar del entorno de aprendizaje por refuerzo.
Stable-Baselines3	Implementación de PPO y utilidades de entrenamiento.
TensorBoard	Monitorización de escalares y evolución de métricas durante ejecuciones PPO.
RViz2	Visualización cualitativa del UAV, el objetivo y la trayectoria en el espacio simulado.

Cuadro 4.3: Herramientas principales utilizadas durante la implementación y validación.

4.5. Implementación del entorno de aprendizaje

La clase `AS2TestEnv` materializa el diseño del Capítulo 3. En lugar de volver a justificar los espacios de observación y acción, esta sección se centra en cómo se ejecuta el contrato en el código. El entorno registra `AS2TestEnv-v0`, define los espacios `Box` correspondientes, aplica los límites configurados y encapsula las operaciones de ciclo de vida que requiere `Aerostack2`.

Durante `reset`, el entorno prepara la interfaz robótica, arma el UAV, activa el modo *offboard*, ejecuta el despegue y devuelve la primera observación normalizada. Cuando se habilita `randomize_hover_start`, la posición inicial, la guiñada y el objetivo se muestrean dentro de los límites configurados. `Exp011` añade márgenes en XY y altura, junto con `max_start_target_distance`, para evitar episodios que comiencen junto a fronteras problemáticas o con la observación saturada desde el primer paso.

Durante `step`, la acción recibida se recorta mediante `max_vel` y `max_yaw_vel`, se envía a `Aerostack2` con `SpeedMotion` y se lee el estado actualizado del UAV. Con esa información se calcula la recompensa, se evalúan las condiciones de terminación y se generan campos de diagnóstico.

La Figura 4.3 resume el flujo implementado. La responsabilidad principal del entorno no es definir de nuevo el problema de aprendizaje, sino convertir cada interacción de PPO en una operación robótica acotada, observable y registrable.

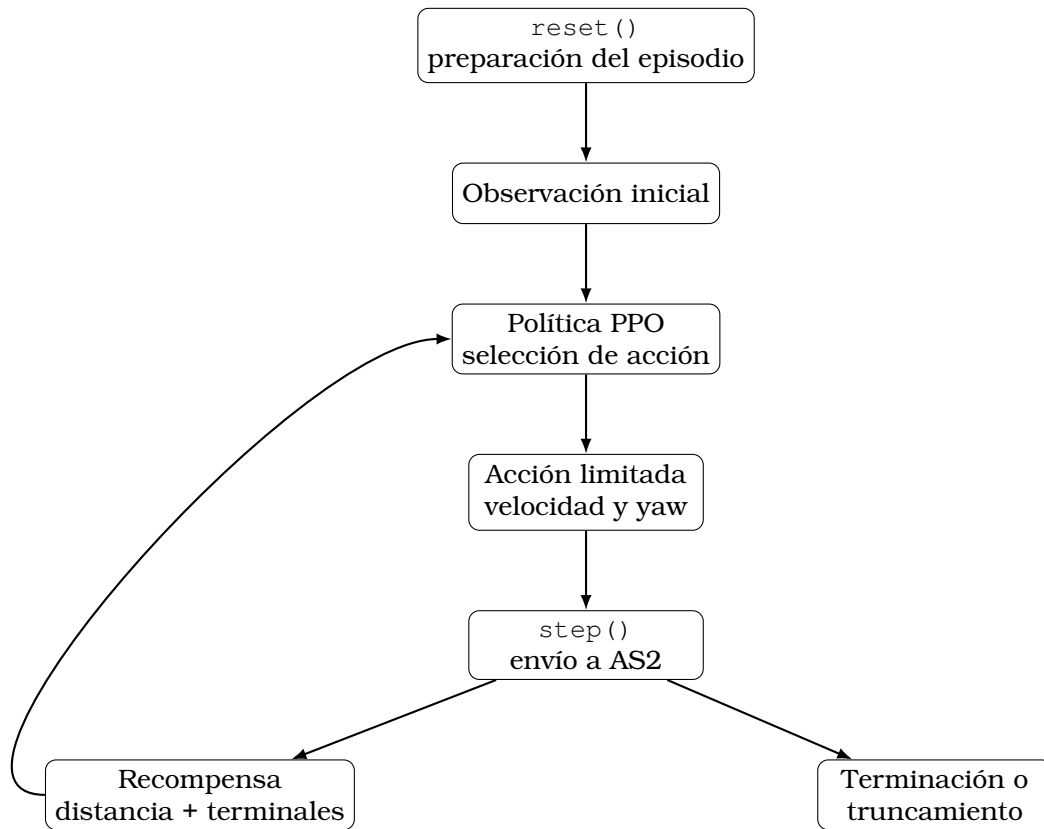


Figura 4.3: Ciclo implementado de un episodio de aprendizaje por refuerzo sobre AS2TestEnv.

4.6. Diagnóstico del reinicio y de la accionabilidad

La implementación añade un paso específico porque el entrenamiento solo es válido si el UAV parte de un estado físico coherente y responde a las acciones enviadas. Por este motivo, el entorno registra el método de reinicio, el camino seguido, el estado del servicio, los errores residuales de posición y guiñada, el desplazamiento físico, la longitud de trayectoria, la aceptación de comandos y la altitud mínima. Estos campos permiten distinguir un fallo de aprendizaje de un fallo previo de simulación o de accionabilidad.

El parámetro `reset_max_vel` separa la autoridad usada por los comandos internos de reinicio de la velocidad máxima permitida a PPO. Por otra parte cabe destacar que cuando la interfaz, acepta comandos pero el UAV no se desplaza de forma apreciable, el entorno falla de forma cerrada y evita generar experiencia inválida.

El servicio `ResetSimulatorState`, añadido en el fork del simulador, aplica una pose objetivo y reinicia velocidades, aceleraciones, fuerzas, pares, motores, controlador, IMU y odometría. Sobre ese servicio se implementan reintentos mediante `reset_service_max_attempts` y `reset_service_retry_backoff_s`. Exp010 mostró que abortar tras un fallo transitorio era demasiado frágil; Exp011 añadió reintentos, y Exp012 redujo la frecuencia de reinicios forzados mediante un guard de acción en fronteras.

El guard de Exp012 modifica las acciones que empujan al UAV hacia fuera cuando se

aproxima a los límites XY o al techo de la escena. Las terminaciones por salida de límites y baja altitud se mantienen como red de seguridad, pero dejan de ser el mecanismo habitual para corregir a un agente aún no entrenado. A partir de Exp013 el disparo de ambos guards es predictivo: se evalúa la posición estimada como posición actual más velocidad medida por un horizonte configurable (`bounds_guard_lookahead_s`, `low_altitude_guard_lookahead_s`), y el empuje vertical dispone de autoridad completa (`bounds_guard_vertical_push_speed`), porque los datos de Exp012 mostraron que la inercia vertical atravesaba un guard puramente reactivo (404 de 428 salidas de límites se produjeron por techo o suelo).

Para la ejecución vectorizada se implementa además el mecanismo de vigilancia de comandos inactivos descrito en el diseño. Un hilo de fondo comprueba el tiempo transcurrido desde la última referencia de movimiento enviada por cualquier vía (pasos del agente, comandos de reinicio o sondas de accionabilidad); si el vehículo está en vuelo y ese tiempo supera `idle_command_watchdog_s`, publica referencias de velocidad nula a la cadencia de publicación configurada hasta que el flujo normal se reanuda. La validación en vivo del mecanismo redujo la deriva por referencias persistentes de 8,05 m a 0,27 m en una ventana de 20 s sin comandos. El parámetro vale cero por defecto, de modo que la ejecución monodron no crea el hilo ni altera su comportamiento.

4.7. Infraestructura de entrenamiento y vectorización

La infraestructura de entrenamiento se concentra en `ppo_setup.py`. Este módulo carga la configuración, crea los directorios de salida, instancia los entornos, envuelve la ejecución con `VecMonitor` y construye el modelo PPO. El registro de métricas se realiza sobre el entorno vectorizado para obtener estadísticas de episodio agregadas sin duplicar lógica de monitorización en cada instancia.

La vectorización se implementa mediante espacios de nombres ROS 2. A partir del prefijo `drone` se generan instancias como `drone0`, `drone1`, `drone2`, ..., `droneN`, todas conectadas al mismo simulador compartido. Esta solución evita lanzar un simulador por entorno y mantiene la correspondencia entre instancia de Gymnasium y UAV simulado. La implementación puede usar `DummyVecEnv` o `SubprocVecEnv` según la configuración, aunque la validación inicial prioriza ejecuciones controladas antes que paralelización agresiva.

El entrenamiento se ejecuta en dos pasos explícitos. Primero se lanza o verifica el simulador con los scripts de AS2; después se ejecuta `scripts/train_ppo.py` con la configuración seleccionada. Para campañas largas, `train_ppo.py` acepta `-resume-from`, que reanuda desde un *checkpoint* explícito o desde el más reciente y calcula el número de pasos restante hasta el total configurado (Stable-Baselines3 suma el argumento al contador del *checkpoint*, por lo que pasar el total completo duplicaría el objetivo en cada reanudación).

El supervisor `scripts/supervise_train.sh` automatiza tres mecanismos sobre esa base. Ante un fallo del entrenamiento, reinicia el stack AS2 completo, espera a que el servicio de reinicio de cada dron esté disponible y reanuda desde el último *checkpoint*. Mediante la variable `RECYCLE_SECONDS` aplica además un reciclado preventivo donde interrumpe el entrenamiento a intervalos fijos y lo reanuda sobre un stack recién iniciado, de modo que cada tramo se ejecuta sobre un simulador sin degrada-

4.7. Infraestructura de entrenamiento y vectorización

ción acumulada; los reciclados se detectan por el tiempo transcurrido, no solo por el código de salida, y no consumen el presupuesto de reintentos por fallo. Por último, `DRONE_NAMESPACES` extiende el reinicio y la comprobación de salud a varios drones para las campañas vectorizadas, e `INITIAL_RESUME` permite sembrar una etapa del curriculum con el modelo final de la etapa anterior cuando aún no existen *check-points* propios.

La validación inicial confirma que el pipeline puede crear el modelo PPO, conectarse con AS2, enviar comandos de velocidad y generar trazas monitorizables. Esta evidencia prueba la infraestructura de entrenamiento, no el rendimiento final de la política; esa interpretación corresponde a los capítulos de pruebas y resultados.

Capítulo 5

Pruebas y evaluación

Este capítulo define la estrategia de validación del sistema desarrollado. Las pruebas no se limitan a comprobar que el software se ejecuta, sino que permiten valorar si la interfaz entre simulación, *middleware* robótico y entorno de aprendizaje proporciona observaciones, acciones, recompensas y condiciones de reinicio coherentes. En un sistema de aprendizaje por refuerzo para UAV, esta validación previa es imprescindible: si el episodio no se reinicia correctamente o si una acción aceptada por la interfaz no produce movimiento físico, los datos de entrenamiento dejan de representar la tarea que se pretende aprender.

5.1. Estrategia de validación

La estrategia de pruebas se organiza en cuatro niveles. El primer nivel agrupa pruebas deterministas del entorno ejecutadas sin depender del simulador real. Estas pruebas verifican cálculo de observaciones, recompensas, terminaciones, recorte de acciones y propagación de configuración. El segundo nivel reúne validaciones de integración con Aerostack2 y ROS 2, cuyo objetivo es comprobar que el entorno puede iniciar el UAV, enviar comandos y recibir estado físico del simulador. El tercer nivel corresponde a ejecuciones PPO acotadas, utilizadas para comprobar que la infraestructura de entrenamiento genera métricas, *checkpoints* y diagnósticos interpretables. El cuarto nivel es la evaluación formal: campañas de 100 episodios generados por semillas fijas, idénticos entre controladores, que producen las métricas comparativas del Capítulo 6.

La validación se plantea de forma incremental. Antes de lanzar entrenamientos de mayor duración se exige que el entorno pase pruebas de configuración, que el simulador se encuentre en un estado limpio y que los reinicios de episodio sean físicamente válidos. Esta decisión evita confundir un problema de aprendizaje con un problema de infraestructura. Los experimentos que terminan en fallo se mantienen como evidencia útil cuando permiten identificar una condición de diseño o de integración que debe corregirse antes de continuar. El Cuadro 5.1 resume los niveles de validación aplicados y la evidencia esperada en cada uno de ellos.

Nivel	Objetivo	Evidencia esperada
Pruebas deterministas	Validar la semántica del entorno sin depender del estado del simulador.	Observaciones, recompensas, terminaciones y parámetros reproducibles.
Pruebas de integración AS2	Verificar conexión, ciclo de vida del UAV, reinicio y comandos de velocidad.	Movimiento físico medible, diagnósticos de reset, servicio de reinicio disponible y ausencia de bloqueos indefinidos.
Ejecuciones acotadas	PPO Comprobar que la infraestructura de entrenamiento produce métricas y artefactos.	Ficheros de monitor, TensorBoard, <i>checkpoints</i> y clasificación de fallos.

Cuadro 5.1: Niveles de validación aplicados al entorno desarrollado.

5.2. Entorno experimental

Las pruebas se realizan sobre el repositorio `rl_uav_aerostack2` mencionado anteriormente en este trabajo, el cual contiene el entorno `AS2TestEnv-v0`, las configuraciones PPO, los scripts de entrenamiento y las utilidades de validación. La ejecución con simulador se apoya en ROS 2 Humble, Aerostack2 y el AS2 Multirrotor Simulator. La ejecución de los scripts Python se organiza mediante el entorno de ejecución del proyecto, que incluye Gymnasium, Stable-Baselines3 y las dependencias de comunicación con Aerostack2.

La observabilidad de las ejecuciones se apoya en varias fuentes. Los ficheros de monitor registran información por episodio, TensorBoard permite inspeccionar métricas agregadas cuando PPO alcanza suficientes pasos de *rollout*, y RViz2 proporciona una vista cualitativa del estado del UAV, el marcador de objetivo y la trayectoria reciente. Además, el entorno incorpora campos específicos de diagnóstico para reset y accionabilidad, de modo que una ejecución fallida pueda clasificarse sin depender únicamente de la inspección visual.

5.3. Pruebas deterministas y de configuración

Las pruebas deterministas verifican las propiedades del entorno que no requieren un simulador activo. Entre ellas se incluyen la forma y rango de la observación, la acotación de acciones, la propagación de parámetros desde YAML, la lógica de éxito, salida de límites y truncamiento, y la compatibilidad con entornos vectorizados. Este nivel de prueba es necesario porque permite detectar errores de formulación antes de introducir la variabilidad propia de ROS 2 y del simulador físico.

Pruebas y evaluación

Validador	Propósito	Criterio de aceptación
<code>scripts/test_vectorization.py</code>	Comprobar contrato del entorno, vectorización, reset y diagnósticos con dobles de prueba.	Todas las comprobaciones deterministas pasan.
<code>scripts/test_ppo_config_propagation.py</code>	Verificar que los parámetros YAML llegan al entorno y al modelo PPO.	Los parámetros críticos aparecen en la instancia final.
<code>scripts/test_exp010_reward_contract.py</code>	Comprobar la recompensa mínima usada en la configuración alineada con el tutor.	La recompensa continua depende de la distancia normalizada y el éxito aplica la penalización terminal de guiñada.
<code>scripts/test_exp011_randomization_contract.py</code>	Validar muestreo aleatorio de inicio, objetivo y guiñada con margen de seguridad y límite de distancia.	Las muestras respetan separación mínima, distancia máxima y márgenes de escena.
<code>scripts/test_exp012_training_contract.py</code>	Verificar la configuración del guard de límites, el corredor vertical y las claves de monitorización.	La configuración de Exp012 conserva la recompensa del tutor y activa las protecciones de frontera.
<code>scripts/test_resume_checkpoint.py</code>	Validar la resolución de <i>checkpoints</i> para reanudación de PPO.	El entrenamiento puede reanudarse desde una ruta explícita o desde el último <i>checkpoint</i> .
<code>scripts/test_bounds_action_guard.py</code> y <code>scripts/test_predictive_guard.py</code>	Verificar los guards de acción reactivos y predictivos en fronteras y techo.	Los comandos salientes se sustituyen por empuje interior en las condiciones geométricas especificadas.
<code>scripts/test_idle_watchdog.py</code>	Validar el mecanismo de vigilancia de comandos inactivos.	Publica hover tras el umbral de inactividad, no interfiere con la operación normal y se desactiva por defecto.
<code>scripts/test_evaluation_harness.py</code>	Verificar el protocolo de evaluación por semillas y sus métricas.	Dos evaluaciones con la misma semilla producen episodios idénticos y los artefactos CSV/JSON son coherentes.
<code>scripts/test_pid_controller.py</code>	Validar el controlador PID de referencia.	Signos, saturación, anti- <i>windup</i> y convergencia en lazo cerrado cinemático.
<code>scripts/test_exp01*_training_contract.py</code>	Fijar el contrato de cada configuración experimental, Exp013–Exp018.	Geometría de guards, curriculum e hiperparámetros del tutor invariantes entre etapas.
<code>scripts/validate_deterministic_reset.py</code>	Validar reinicios controlados y coherencia de pose inicial.	El UAV o doble de prueba alcanza el estado inicial esperado.
<code>scripts/validate_exp008_training_smoke.py</code>	Ejecutar una prueba corta con acción fija o PPO y analizar movimiento físico.	Las acciones aceptadas producen desplazamiento suficiente y métricas coherentes.

Cuadro 5.2: Validadores representativos utilizados durante el desarrollo.

El Cuadro 5.2 resume los validadores más relevantes. La existencia de estas pruebas permite tratar el entrenamiento PPO como una fase posterior a la validación del

entorno, no como el único mecanismo para detectar fallos.

5.4. Pruebas con Aerostack2 y reinicio de episodios

Las pruebas con Aerostack2 evalúan si el contrato de Gymnasium puede sostenerse sobre un sistema robótico simulado. En particular, se comprueba que el método `reset` deja el UAV en una pose inicial válida, que `step` envía comandos de velocidad y que `close` no deja referencias activas ni estados bloqueantes. Durante las ejecuciones se detectaron fallos de `reset` que no podían tratarse como simples errores de PPO: episodios que comenzaban fuera de límites, comportamientos bloqueantes de retorno a pose inicial, pérdidas de estabilidad tras recrear el controlador y comandos aceptados sin movimiento físico medible.

Como consecuencia de estos fallos, el reinicio de episodios se trasladó parcialmente al simulador mediante el servicio específico del fork del AS2 Multirotor Simulator que se ha comentado durante la implementación. Este servicio recibe la pose objetivo del UAV, muta el estado interno de la dinámica, limpia velocidades, aceleraciones, referencias y estado de actuadores, y sincroniza la capa de plataforma de Aerostack2 para que el vehículo quede armado, en modo *offboard* y en estado de vuelo. La validación de dicho servicio distingue entre dos fronteras de prueba: la respuesta inmediata del servicio, que informa errores de posición, guiñada y normas dinámicas residuales, y la telemetría posterior publicada por ROS 2, que debe confirmar que no aparecen picos transitorios tras el reinicio.

Para evitar que estos casos contaminen el entrenamiento, el entorno registra razones de fallo y adopta una política de fallo acotado. Si el UAV no vuelve a la pose inicial dentro del tiempo configurado, si no se mantiene estable tras el `reset` o si una acción aceptada no produce movimiento observable, la ejecución se rechaza como evidencia de entrenamiento válido. Este criterio es más estricto que permitir que PPO continúe acumulando pasos, pero resulta necesario para mantener la validez de los datos. El Cuadro 5.3 se incluye en esta sección para vincular cada fallo de integración observado con su causa probable, la mitigación incorporada y la evidencia experimental que justificó el cambio; de este modo, la evolución del entorno queda presentada como un proceso de validación técnica y no como una simple sucesión de intentos de entrenamiento.

Pruebas y evaluación

Fallo observado	Causa probable	Mitigación incorporada	Evidencia
Éxitos triviales de un paso	Objetivo demasiado cercano al despegue inicial.	Objetivos desplazados y separación explícita entre inicio y objetivo.	Exp001–Exp002.
Bucle de episodios fuera de límites	Reset fijo insuficiente tras una salida de escena.	<code>fixed_start_pose</code> y retorno controlado a pose inicial.	Exp002–Exp003.
Bloqueo por <code>go_to</code>	Comportamiento de <code>Aerostack2</code> ejecutándose indefinidamente.	Eliminación del fallback bloqueante y fallo explícito.	Exp006–Exp007.
Timeout de reset por velocidad	Derivas grandes o recuperación insuficiente entre episodios.	Diagnósticos de reset, <code>reset_max_vel</code> y fases de corrección.	Exp007–Exp008a.
Comando aceptado sin movimiento	Desacoplamiento entre aceptación de API y acción física tras reset.	Prueba de accionabilidad y fallo cerrado antes de entrenar.	Smokes Exp008a.
Degradación por reinicios de servicio repetidos	Muchos episodios terminan en fronteras no observables y fuerzan teleports sucesivos.	Reintentos completos del servicio, guard de acción en límites y supervisor con reanudación.	Exp010–Exp012.
Inercia vertical atraviesa el guard reactivo	La plataforma invierte la velocidad vertical con lentitud y el margen se cruza antes de que actúe la corrección.	Disparo predictivo (posición + velocidad $\times$ horizonte) y empuje vertical con autoridad completa.	Exp012–Exp013.
Deriva en vectorizado por <code>lockstep</code>	El controlador mantiene la última referencia de velocidad mientras el paso vectorizado espera el reset de otro entorno (8,05 m en 20 s medidos).	Watchdog de comandos inactivos con consigna de hover periódica.	Exp017–Exp018, sondas dedicadas.
Cuelgue del reciclado preventivo	La señal de terminación puede coincidir con un reset en curso y el cierre de ROS 2 queda bloqueado.	Escalado a terminación forzosa tras un tiempo de gracia en el supervisor.	Exp016.

Cuadro 5.3: Fallos de integración identificados y mecanismos de control incorporados.

Además de las pruebas ejecutadas desde el repositorio del entorno, el fork del simulador incorpora pruebas unitarias sobre el servicio de reinicio. Estas pruebas verifican que el servicio está disponible, que rechaza entradas no finitas o tolerancias inválidas, que limpia referencias dinámicas obsoletas y que mantiene una altitud válida incluso cuando el estado previo del vehículo no era apto para iniciar un nuevo episodio. Estas comprobaciones son relevantes porque el servicio no solo debe modificar la pose externa del UAV, sino también dejar el estado interno del simulador en condiciones compatibles con el siguiente comando de velocidad.

5.5. Métricas y criterios de aceptación

Las métricas empleadas combinan indicadores de aprendizaje e indicadores de infraestructura. En el primer grupo se encuentran la recompensa por episodio, la longitud de episodio, la tasa de éxito y la distancia final al objetivo. En el segundo grupo se registran razones de terminación, desplazamiento físico, longitud de trayectoria, altitud mínima, ratio de comandos aceptados, método de reset y clase de fallo. Esta separación permite determinar si una ejecución fracasa porque la política todavía no resuelve la tarea o porque el entorno no está entregando transiciones físicamente válidas.

Las curvas de TensorBoard se utilizan como evidencia complementaria, no como única prueba de aprendizaje. En particular, las métricas `rollout/ep_rew_mean` y `rollout/ep_len_mean` permiten observar tendencias agregadas, mientras que los campos de monitor por episodio permiten explicar esas tendencias mediante razones de terminación, distancia final, éxitos, salidas de límites o eventos de baja altitud. Esta combinación es necesaria porque una curva plana de recompensa puede deberse a una política que no aprende, pero también a episodios dominados por terminaciones de seguridad o por reinicios costosos del simulador.

El criterio de aceptación para un entrenamiento no se reduce a que el proceso de Python siga ejecutándose. Una ejecución solo se considera apta para análisis de aprendizaje si el entorno inicia episodios desde estados válidos, si las acciones producen movimiento físico medible, si los fallos de reset no se ocultan como episodios válidos y si las métricas de monitorización permiten reconstruir lo ocurrido. En caso contrario, el resultado se documenta como evidencia de integración y se utiliza para mejorar el entorno antes de extraer conclusiones sobre PPO.

Capítulo 6

Resultados

Este capítulo presenta los resultados obtenidos mediante la estrategia de pruebas definida previamente. La exposición distingue entre resultados de infraestructura y resultados de aprendizaje, y sigue la evolución experimental completa: primero la validación funcional, después la campaña de experimentos que condujo desde un aprendizaje nulo hasta una política con tasa de éxito del 100 % en evaluación determinista, a continuación la evaluación formal frente a un controlador PID de referencia y, por último, la validación del entrenamiento vectorizado con dos drones sobre un mismo simulador.

6.1. Validación funcional de la infraestructura

Las primeras pruebas confirmaron que la infraestructura básica era operativa. El entorno puede instanciarse como entorno Gymnasium, conectarse con Aerostack2, armar el UAV, activar el modo *offboard*, ejecutar despegue, enviar comandos de velocidad y registrar métricas mediante los mecanismos de monitorización. Esta validación funcional demuestra que la integración entre Gymnasium, Stable-Baselines3, ROS 2 y Aerostack2 es viable.

No obstante, los resultados también mostraron que la validez de un entrenamiento depende de condiciones adicionales. En las primeras ejecuciones se observaron éxitos triviales cuando el objetivo estaba demasiado cerca de la pose de despegue, ausencia de escalares de TensorBoard cuando PPO no alcanzaba suficientes pasos de *rollout*, y bucles de episodios fuera de límites cuando el reset no garantizaba un estado inicial seguro. Las iteraciones posteriores permitieron certificar el reinicio mediante el servicio del simulador y ejecutar entrenamientos más largos, pero también revelaron que las salidas de límites y la degradación acumulada por reinicios de servicio podían dominar el comportamiento observado. Estos resultados condujeron a separar el problema en dos niveles: primero validar el ciclo físico de episodio y después analizar el aprendizaje.

6.2. Matriz de experimentos PPO

El Cuadro 6.1 resume la evolución de los experimentos principales. La tabla no debe interpretarse como una búsqueda lineal de hiperparámetros, sino como una secuen-

6.2. Matriz de experimentos PPO

cia de diagnóstico en la que cada ejecución revela una propiedad de la tarea o de la infraestructura y motiva el siguiente cambio de diseño.

Experimento	Objetivo	Resultado observado	Interpretación técnica
Exp001	PPO con configuración inicial.	Episodios de éxito en un paso y métricas insuficientes.	El objetivo estaba demasiado cerca del estado inicial; no era una tarea de navegación útil.
Exp002	Objetivo desplazado y TensorBoard en prueba corta.	Primeros episodios útiles, después bucles de salida de límites.	El pipeline generaba artefactos, pero el reset no garantizaba inicio seguro tras OOB ¹ .
Exp003	Introducir <code>fixed_start_pose</code> .	Entrenamiento corto completado con monitor, TensorBoard y <code>checkpoints</code> .	El reset determinista resolvió el bucle inmediato de OOB en una prueba acotada.
Exp004–Exp008a	Estabilización incremental del reset.	Bloqueos de <code>go_to</code> , timeouts de reset y comandos aceptados sin movimiento.	El cuello de botella pasa de PPO a la interacción reset–controlador–simulador.
Exp009	Primer gate PPO sobre el stack de reset certificado.	5000 pasos y 126 episodios sin fallos de reset.	Se valida la infraestructura de episodio; el aprendizaje aún no es interpretable.
Exp010	Modificación a recompensa mas sencilla, randomización completa e hiperparámetros revisados.	12 852 pasos y 427 episodios antes del fallo; 359 OOB, 65 baja altitud, 3 éxitos.	Las fronteras no observables y el coste del teleport dominan la experiencia; el simulador se bloquea tras cientos de reinicios de servicio.
Exp011	Área segura de muestreo y límite de distancia inicio–objetivo.	21 347 pasos y 637 episodios; 512 OOB, 121 baja altitud, 3 éxitos.	La saturación de observaciones se corrige, pero una política sin entrenar deriva igualmente hasta las fronteras invisibles.
Exp012	Guard de acción en límites, corredor vertical ampliado, reanudación y supervisor.	704 episodios; primer aprendizaje real, con picos del 24 % de éxito en simulador recién reiniciado.	El guard XY funciona (24 muertes frente a 512), pero la inercia vertical atraviesa el guard reactivo (404 muertes por techo y suelo) y la degradación del simulador es gradual: la misma política pasa del 9 % al 1 % según envejece el proceso.
Exp013	Guards predictivos (posición + velocidad $\times 1$ s) y empuje vertical con autoridad completa.	Primer entrenamiento de 50 000 pasos completado sin ningún fallo; 0 teleports en los primeros 24 000 pasos.	La infraestructura queda validada por completo; el aprendizaje permanece plano (~ 11 %), lo que identifica la densidad de la señal de éxito como cuello de botella.
Exp014	Curriculum etapa 1: objetivos a $\leq 2,5$ m.	27 % de éxito global con tendencia creciente (picos del 50 %).	Primera curva de aprendizaje sostenida del trabajo: con éxito alcanzable, el gradiente arranca.
Exp015	Curriculum etapa 2: objetivos a $\leq 3,5$ m con warm start.	65 % de éxito global, picos del 90 %.	La política transfiere lo aprendido a distancias mayores sin olvidar los objetivos cercanos.
Exp016	Curriculum etapa 3: rango completo de 5 m (tarea original).	91 % global; 99 % (158/160) en el tramo final sobre simulador recién reiniciado.	La tarea completa queda resuelta con la recompensa y los hiperparámetros indicados por el tutor.
Exp017	Primer entrenamiento vectorizado con 2 drones.	Colapso inmediato: 1184/1189 episodios OOB con muertes a los 2 pasos.	El diagnóstico identifica la causa raíz: el controlador mantiene la última referencia de velocidad durante los resets bloqueantes del otro entorno (deriva medida de 8,05 m en 20 s).
Exp018	Vectorizado con watchdog de comandos inactivos.	96 % de éxito global en 1272 episodios; 100 % (244/244) en el tramo final.	El entrenamiento vectorizado queda validado con duplicación real del muestreo y mejora adicional de la política.

Cuadro 6.1: Resumen de experimentos principales y su interpretación.

La Figura 6.1 sintetiza esta evolución en términos de tasa de éxito por experimento. Los tres primeros experimentos de la fase alineada con el tutor (Exp010–Exp012) per-

¹OOB corresponde a *out of bounds*, es decir, una terminación por salida de los límites físicos definidos para el episodio.

Resultados

manecen por debajo del 5 % pese a que la infraestructura mejora en cada iteración; el salto se produce con el curriculum de distancia (Exp014–Exp016) y culmina con el entrenamiento vectorizado (Exp018).

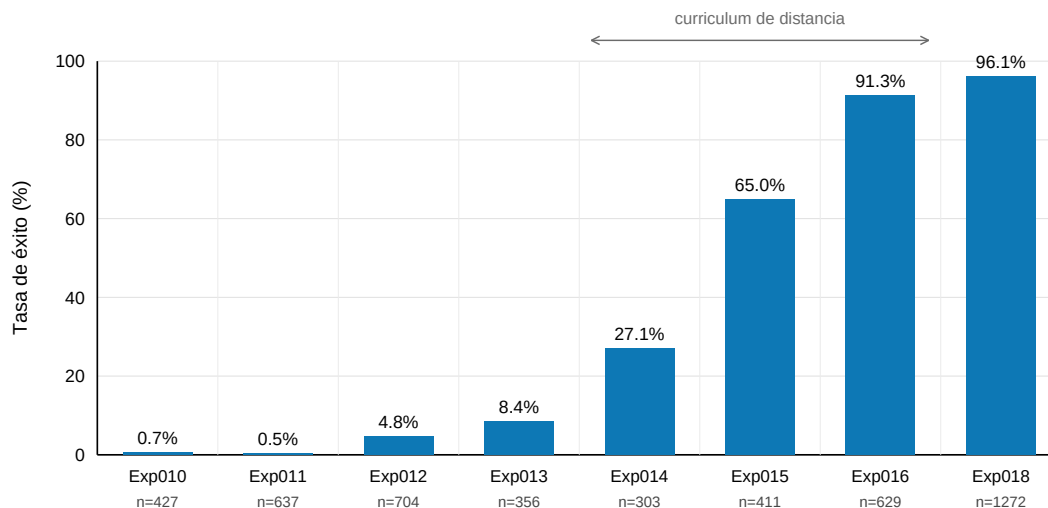


Figura 6.1: Tasa de éxito por experimento durante la campaña principal. Cada barra agrega todos los episodios registrados por el monitor vectorizado del experimento; n indica el número de episodios.

6.3. Aprendizaje mediante curriculum de distancia

Los experimentos Exp010–Exp013 demuestran que, con la recompensa mínima $-d_{\text{norm}}$ y éxito escaso (alcanzar una esfera de 0,4 m desde distancias de hasta 5 m), la exploración aleatoria no encuentra suficiente señal de éxito para arrancar el gradiente con la tasa de aprendizaje configurada. Una vez eliminadas todas las causas de invalidez física (Exp013 completa 50 000 pasos sin un solo fallo de reset ni teleport durante sus primeros 24 000 pasos), la tasa de éxito permanece plana en torno al 11 %, lo que permite atribuir el estancamiento a la definición de la tarea y no a la infraestructura.

La solución adoptada es un curriculum de distancia en tres etapas con *warm start*: la etapa 1 limita la separación inicio–objetivo a 2,5 m, de modo que un vuelo directo requiere unos 25 pasos y la exploración encuentra la esfera de éxito con frecuencia suficiente; las etapas 2 y 3 amplían el límite a 3,5 m y 5,0 m reutilizando como punto de partida el modelo final de la etapa anterior, y mantienen la separación mínima de 1,0 m para que los objetivos cercanos sigan presentes en la distribución de episodios.

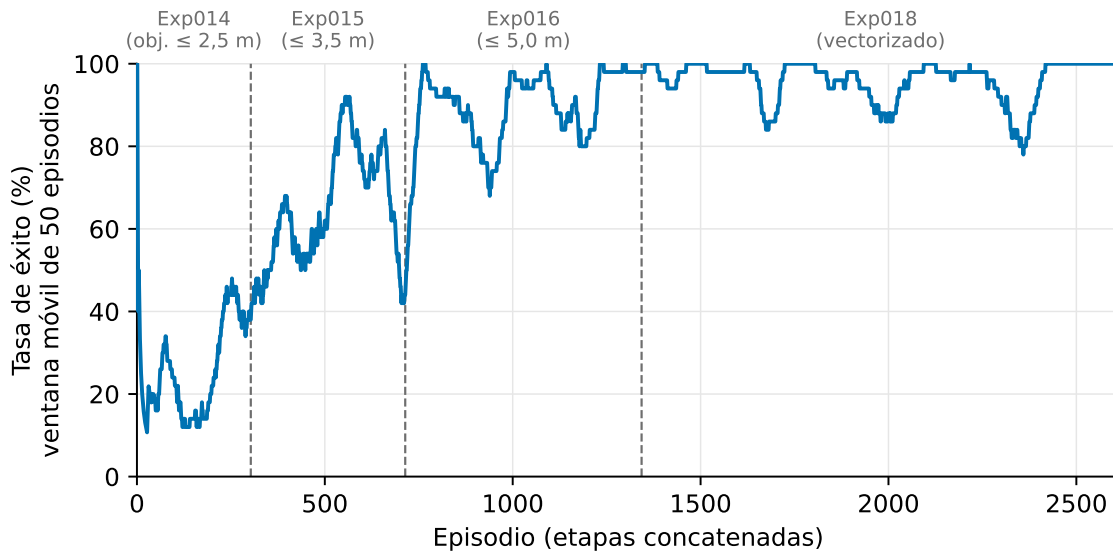


Figura 6.2: Tasa de éxito en ventana móvil de 50 episodios a lo largo de las tres etapas del currículo y del entrenamiento vectorizado posterior. Las líneas discontinuas marcan los cambios de etapa, en los que se amplía el rango de distancias muestreado.

La Figura 6.2 muestra la curva de aprendizaje resultante. Cada ampliación del rango produce una caída transitoria seguida de recuperación por encima del nivel anterior, el patrón esperado de un currículo bien dimensionado. La recompensa media por episodio, en cambio, no es un indicador fiable de esta mejora: al ser la recompensa continua $-d_{\text{norm}}$ por paso, los episodios largos y exitosos acumulan más penalización total que los episodios cortos que terminaban en fallo, por lo que la media de recompensa puede empeorar mientras la tasa de éxito mejora. Por este motivo el análisis se apoya en la tasa de éxito, la distancia final y la composición de terminaciones.

6.4. Evaluación formal y comparación con un controlador PID

La evaluación formal utiliza el protocolo descrito en el Capítulo 5: 100 episodios generados con semillas fijas (1000–1099), idénticos para todos los controladores evaluados, ejecutados sobre un simulador recién iniciado. Se evalúan la política de la etapa 3 del currículo (Exp016), la política tras el entrenamiento vectorizado (Exp018), ambas en modo determinista y estocástico, y el controlador PID de referencia que consume la misma observación relativa normalizada y emite acciones en el mismo espacio acotado.

La distinción entre modo determinista y estocástico es relevante porque PPO aprende una distribución de acciones, no solo una acción fija. En modo determinista se aplica la media de esa distribución y, por tanto, se evalúa el comportamiento más estable de la política aprendida. En modo estocástico se muestrea una acción de la distribución, conservando parte de la variabilidad usada durante el entrenamiento. Comparar ambos modos permite distinguir si la política ya ha aprendido una acción media suficiente o si todavía depende del ruido de exploración para alcanzar el objetivo.

Resultados

Controlador	Éxito	Dist. final (m)	Pasos (med.)	Tiempo (med.)	Error yaw (rad)
PPO Exp016 determinista	78/100	0,43	30	6,0 s	0,58
PPO Exp016 estocástico	85/100	0,40	37	7,4 s	0,49
PPO Exp018 determinista	100/100	0,32	32	6,4 s	0,44
PPO Exp018 estocástico	89/100	0,36	31	6,2 s	0,49
PID ($k_p = 0,8$)	100/100	0,31	27	5,4 s	0,01

Cuadro 6.2: Comparación de controladores sobre los mismos 100 episodios sembrados. Pasos y tiempo corresponden a la mediana de los episodios con éxito; el error de yaw es la media absoluta en el instante terminal.

El Cuadro 6.2 y la Figura 6.3 recogen los resultados. Tres observaciones estructuran el análisis.

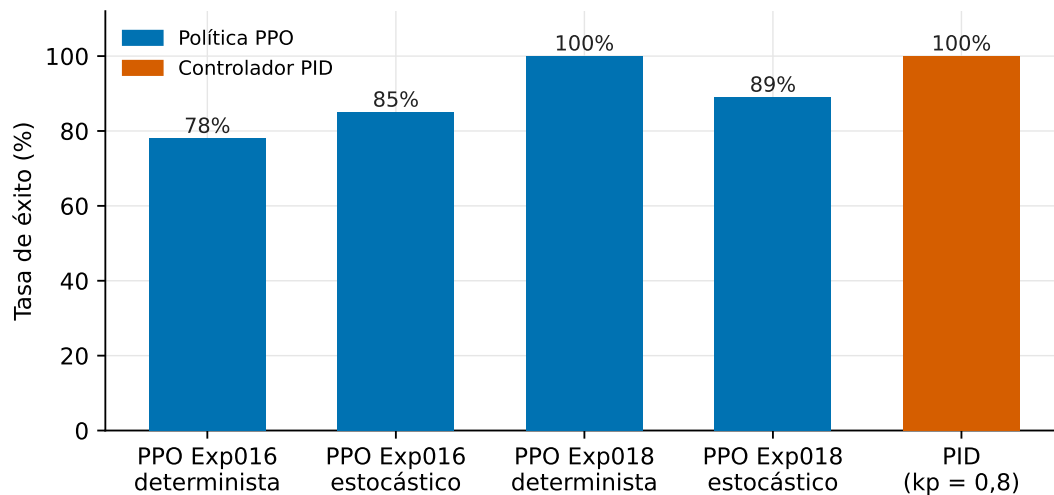


Figura 6.3: Tasa de éxito de los cinco controladores evaluados sobre los mismos 100 episodios sembrados.

En primer lugar, el entrenamiento vectorizado mejora la política. El modelo de Exp016 alcanza un 78 % en modo determinista, con un modo de fallo característico: los 22 episodios fallidos terminan todos por límite de pasos con el UAV detenido entre 0,42 y 1,12 m del objetivo, sin ninguna terminación de seguridad. La Figura 6.4 muestra esta distribución. Tras 50 000 pasos adicionales de entrenamiento vectorizado (Exp018), este estancamiento cerca del objetivo desaparece por completo y la política resuelve los 100 episodios.

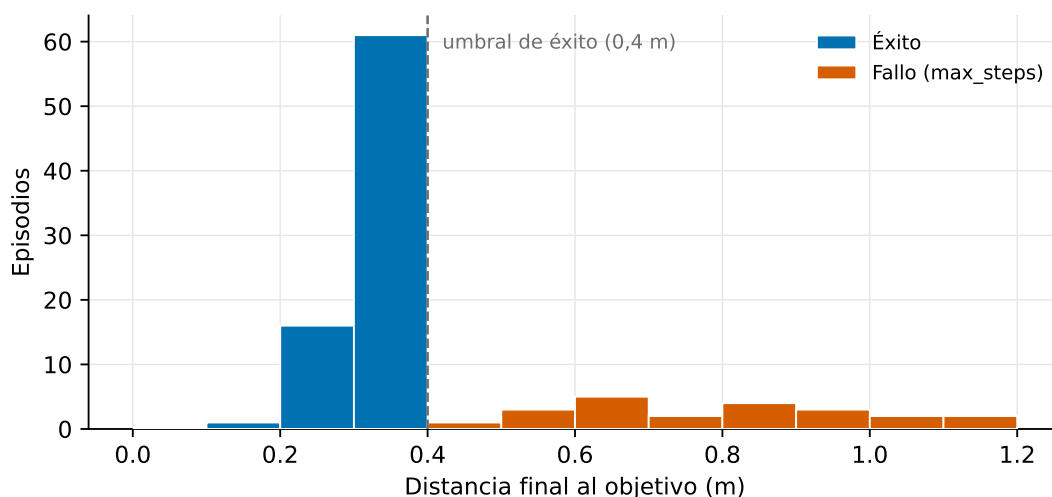


Figura 6.4: Distribución de la distancia final al objetivo en la evaluación determinista del modelo Exp016. La línea discontinua indica el umbral de éxito de 0,4 m. Todos los fallos corresponden a truncamientos por límite de pasos, sin terminaciones de seguridad; una parte de ellos queda próxima al umbral, aunque existen episodios con distancias finales superiores.

En segundo lugar, la relación entre los modos determinista y estocástico se invierte con la madurez de la política. En Exp016 el muestreo estocástico mejora el resultado (del 78 % al 85 %), porque el ruido de exploración empuja al UAV a cruzar la frontera de la esfera de éxito cuando la acción media se estanca. En Exp018, con la política ya afinada, el mismo ruido solo introduce riesgo y el modo estocástico rinde peor (89 % frente a 100 %). Este comportamiento ilustra la transición de una política que depende de la exploración a una política que la ha internalizado.

En tercer lugar, el controlador PID gana la comparación en la tarea nominal: con la misma tasa de éxito que la mejor política, resuelve los episodios un 16 % más rápido, con trayectorias prácticamente rectas y con un error de yaw terminal de 0,01 rad frente a 0,44 rad. Este resultado es esperable, la contribución del aprendizaje por refuerzo en este trabajo no consiste en superar al PID en su terreno, sino en alcanzar la paridad de éxito con el óptimo clásico mediante una política aprendida de extremo a extremo, sobre una infraestructura que permite abordar después tareas donde no existe una ley de control evidente. El error de yaw residual de la política tiene además una explicación de diseño: la bonificación terminal $R_{\text{success}} - w_{\text{yaw}}|e_{\text{yaw}}|/\pi$ sigue siendo positiva con yaw deficiente, por lo que la política aprende a llegar sin refinar la orientación final.

6.5. Validación del entrenamiento vectorizado

La validación de la ejecución vectorizada aporta dos resultados: una medición y un diagnóstico. La medición confirma que, con `SubprocVecEnv` y dos drones en el mismo simulador, cada paso vectorizado se completa en 0,20 s, el mismo tiempo que un paso individual. En ese paso se obtiene una transición de cada dron, por lo que el caudal efectivo de muestras se duplica respecto a la ejecución con un único entorno. La alternativa secuencial no aporta aceleración en este sistema al ser el simulador de

reloj real, N entornos secuenciales tardan $N \times 0,2$ s en producir N muestras, exactamente el mismo caudal que un único entorno, independientemente del beneficio de la inferencia por lotes.

El diagnóstico corresponde al fallo de Exp017. El primer entrenamiento vectorizado colapsó de inmediato (1184 de 1189 episodios fuera de límites, con mediana de terminación a los 2 pasos y desplazamientos físicamente imposibles de 4 m entre pasos consecutivos). Tres pruebas discriminatorias aislaron la causa: un dron volando en solitario en el mundo multidron se comporta con normalidad; dos drones volando simultáneamente con cadencias independientes también; pero un dron al que se deja de comandar durante 20 s deriva 8,05 m, porque el controlador de Aerostack2 mantiene la última referencia de velocidad como consigna activa indefinidamente. En la ejecución vectorizada en *lockstep*, el reinicio certificado de un entorno (10–25 s) bloquea el paso vectorizado y deja al otro dron volando su última velocidad durante todo ese tiempo, lo que encadena salidas de límites en espiral.

La solución ya comentada anteriormente, un watchdog de comandos inactivos que publica una consigna de hover cuando el entorno lleva más de 0,5 s sin emitir referencias, reduce la deriva medida de 8,05 m a 0,27 m. Con esta corrección se ejecuta Exp018: se parte del modelo final de Exp016, se mantiene la misma tarea de rango completo y se reanuda el entrenamiento con dos instancias del entorno, asociadas a los drones `drone0` y `drone1`, dentro de un `SubprocVecEnv`. Cada instancia ejecuta episodios independientes con sus propias semillas y reinicios, pero el algoritmo PPO recibe las muestras de ambas en el mismo proceso de aprendizaje.

De este modo, Exp018 no es una evaluación aislada de la política anterior, sino un entrenamiento adicional sobre la política ya obtenida en Exp016. La diferencia respecto a Exp017 es que, durante los reinicios bloqueantes de un entorno, el watchdog mantiene al otro dron en hover en lugar de dejarlo continuar con la última referencia de velocidad. Con esta condición, Exp018 completa 50 000 pasos adicionales con un 96 % de éxito global y un 100 % en su tramo final, sin ningún fallo real de ejecución. Estos resultados validan el entrenamiento vectorizado con dos drones y acotan las condiciones para escalar: el número de entornos queda limitado por el coste de los reinicios bloqueantes en *lockstep* y por la capacidad de CPU para integrar varias dinámicas en tiempo real, restricciones que desaparecerían con un simulador pausable accesible mediante *bindings* directos.

6.6. Hallazgos sobre el simulador

La sección experimental de este trabajo produjo dos hallazgos sobre el fork del simulador que condicionan cualquier trabajo futuro sobre esta plataforma.

El primero es la degradación acumulativa asociada al servicio de reinicio. Los experimentos Exp010 y Exp011 terminaron con el canal de comandos bloqueado tras 426 y 634 teleports respectivamente, y Exp012 permitió cuantificar que la degradación es gradual y no solo terminal: la misma política pasó del 9 % de éxito sobre simulador recién iniciado al 1 % tras cientos de teleports, y recuperó el 24 % inmediatamente después de reiniciar el proceso del simulador. La mitigación operativa combina tres mecanismos: los guards de acción reducen las terminaciones en frontera (y con ellas los teleports) en dos órdenes de magnitud, el supervisor reinicia el stack completo y reanuda desde el último *checkpoint* cuando un entrenamiento muere, y el reciclado

preventivo reinicia stack y proceso cada 2,5 horas para que cada tramo de entrenamiento se ejecute sobre un simulador sano.

El segundo es el comportamiento de la referencia de velocidad descrito en la sección anterior, el controlador mantiene la última consigna indefinidamente en ausencia de nuevas órdenes. Este comportamiento es razonable para teleoperación, pero resulta peligroso en entrenamiento vectorizado y debe tenerse en cuenta en cualquier integración de aprendizaje por refuerzo sobre Aerostack2.

6.7. Discusión de limitaciones

Los resultados deben interpretarse con varias limitaciones. Primera, las ganancias del controlador PID de referencia se fijaron en valores razonables ($k_p = 0,8$ con saturación en los límites de acción) sin una búsqueda exhaustiva; un ajuste fino podría mejorar sus tiempos, aunque no su tasa de éxito, ya saturada. Segunda, los guards de acción modifican la tarea efectiva que el agente experimenta: las terminaciones de seguridad se mantienen como red final, pero el agente rara vez las alcanza, por lo que la política aprendida no es directamente comparable con la de un entorno sin protecciones. Tercera, la degradación del simulador se mitiga operativamente pero no se corrige en su raíz, que permanece en el nodo de plataforma del fork. Cuarta, todos los resultados proceden de un único equipo y de un simulador de dinámica simplificada; la transferencia a plataformas reales queda fuera del alcance de este trabajo.

Estas limitaciones no invalidan las conclusiones principales: la tarea propuesta queda resuelta con la formulación original del problema de control, la comparación con el controlador clásico se realiza sobre episodios idénticos y reproducibles, y los fallos encontrados están diagnosticados con causa raíz identificada y evidencia cuantitativa.

Capítulo 7

Impacto del trabajo

Este capítulo analiza el impacto potencial del trabajo desde una perspectiva técnica, académica y social. El análisis se centra en las posibilidades que ofrece un entorno de aprendizaje por refuerzo reproducible para UAVs, así como en sus limitaciones y riesgos cuando se traslada desde simulación hacia sistemas robóticos reales.

7.1. Impacto técnico y académico

El impacto más directo se produce en el ecosistema en el que el trabajo se desarrolla. La infraestructura construida (entorno Gymnasium sobre Aerostack2, reinicio certificado de episodios, guards de acción, supervisión de campañas largas, vectorización validada y protocolo de evaluación reproducible) queda publicada en repositorios abiertos y puede reutilizarse por el grupo de investigación CVAR y por cualquier equipo que desee entrenar políticas de control sobre Aerostack2 sin repetir el proceso de diagnóstico documentado en esta memoria. Los hallazgos sobre el simulador, en particular la degradación acumulativa asociada al servicio de reinicio y la persistencia indefinida de las referencias de velocidad, constituyen además información accionable para los mantenedores del proyecto original, ya que afectan a cualquier uso intensivo del simulador más allá del aprendizaje por refuerzo.

En el plano metodológico, el trabajo aporta un ejemplo documentado de evaluación homogénea entre un controlador aprendido y un controlador clásico: mismos episodios generados por semillas, mismas métricas y una lectura explícita de dónde gana cada enfoque. Varias decisiones se tomaron considerando este impacto: la publicación del protocolo completo de evaluación, la conservación de los experimentos fallidos como evidencia y la cuantificación de los fallos de infraestructura en lugar de su omisión. Este material tiene también valor docente, porque documenta una progresión completa en la que la mayor parte de la dificultad no reside en el algoritmo de aprendizaje sino en la validez física del entorno.

7.2. Impacto social e industrial

Los UAVs autónomos son una tecnología habilitadora en inspección de infraestructuras, agricultura de precisión, logística y respuesta a emergencias. Los controladores de navegación local como el entrenado en este trabajo son un componente básico de

esas aplicaciones, y la reducción de la barrera de entrada para desarrollarlos sobre plataformas abiertas contribuye a que estas capacidades no queden restringidas a soluciones propietarias.

Debe mantenerse, no obstante, cautela sobre el alcance real de lo demostrado. Todos los resultados de este trabajo se obtienen en simulación, con una dinámica simplificada y sin perturbaciones externas. La transferencia a vehículos reales exige etapas adicionales de validación (robustez ante viento, ruido de sensores, retardos de comunicación y fallos parciales) que quedan explícitamente fuera del alcance. Presentar resultados de simulación como capacidades operativas sería una extrapolación injustificada, y la memoria evita deliberadamente ese salto.

Capítulo 8

Conclusiones y líneas futuras

Este capítulo valora el grado de cumplimiento de los objetivos planteados en el Capítulo 1 y resume las conclusiones técnicas y metodológicas del trabajo. La discusión se apoya en la evidencia cuantitativa presentada en el Capítulo 6.

8.1. Conclusiones

El objetivo general del trabajo, desarrollar un entorno de aprendizaje por refuerzo sobre el simulador de multirrotores de Aerostack2 y utilizarlo para entrenar y evaluar un controlador de velocidad, se ha alcanzado de forma satisfactoria. La política final resuelve el 100 % de los 100 episodios del protocolo de evaluación determinista sobre la tarea completa (objetivos aleatorios a hasta 5 m con orientación final), con una distancia final media de 0,32 m.

El primer objetivo específico, la familiarización con Aerostack2 y su simulador, se alcanzó de forma natural durante las primeras fases y quedó reflejado en la identificación de los componentes de control, estimación y comportamiento relevantes para el problema. El segundo, la construcción de la interfaz Gymnasium sobre el simulador, se cumplió con un matiz importante que constituye una de las lecciones del trabajo: la interfaz mínima fue rápida de construir, pero convertirla en un entorno *válido* para aprendizaje por refuerzo exigió un esfuerzo muy superior al previsto, concentrado en el reinicio certificado de episodios y en la verificación de accionabilidad.

El tercer objetivo, la validación experimental del ciclo de interacción, se alcanzó mediante una estrategia de tres niveles (pruebas deterministas, integración con el simulador y ejecuciones PPO acotadas) que acabó articulada en una veintena de suites de validación automatizadas. La consecución del cuarto objetivo, el entorno vectorizado, resultó la más difícil: el primer entrenamiento con dos drones colapsó por una interacción no documentada entre el paso vectorizado en *lockstep* y la persistencia de referencias de velocidad del controlador, y solo un diagnóstico sistemático con pruebas discriminatorias permitió identificar la causa raíz (8,05 m de deriva en 20 s sin comandos) y corregirla con un mecanismo de vigilancia de comandos inactivos. Tras la corrección, el entrenamiento vectorizado duplica el muestreo efectivo y mejoró la política final del 78 % al 100 % de éxito determinista.

El quinto objetivo, la definición de un problema de control progresivo, resultó ser la clave del aprendizaje. Con la recompensa mínima y la tarea completa, la exploración

no encontraba señal de éxito suficiente (tasas planas del 11 % aun con la infraestructura ya libre de fallos); el curriculum de distancia en tres etapas con reutilización del modelo anterior elevó la tasa de éxito del 27 % al 65 % y finalmente al 91–99 % sobre la tarea original. El sexto objetivo, el entrenamiento, la evaluación y la comparación con un controlador PID, se cumplió mediante un protocolo de episodios sembrados idénticos para todos los controladores. La comparación es honesta: el PID gana la tarea nominal en tiempo y precisión de orientación, la política alcanza la paridad en tasa de éxito, y la diferencia queda explicada y cuantificada. La extensión jerárquica planteada como séptimo objetivo opcional no se abordó y se recoge como línea futura.

Más allá de los objetivos, el trabajo deja dos conclusiones metodológicas. La primera es que en aprendizaje por refuerzo sobre simuladores robóticos reales la validez física del episodio es la condición que gobierna todo lo demás: la mayor parte del esfuerzo de ingeniería se invirtió en garantizar que cada transición entregada al algoritmo representara de verdad la tarea, y los mecanismos resultantes (reinicio certificado con verificación de accionabilidad, guards de acción predictivos, supervisión con reanudación y reciclado preventivo) son aportaciones reutilizables independientes de la política entrenada. La segunda es el valor de los resultados negativos tratados como evidencia: los fallos de Exp010, Exp011 y Exp017 produjeron los tres hallazgos más importantes del trabajo (la degradación acumulativa del simulador con los reinicios de servicio, la insuficiencia de los guards reactivos frente a la inercia, y la persistencia indefinida de las referencias de velocidad), todos ellos diagnosticados con causa raíz y cuantificación.

El código desarrollado queda publicado en los dos repositorios indicados en el Capítulo 4, con configuraciones, pruebas automatizadas y artefactos de evaluación reproducibles mediante semillas.

8.2. Líneas futuras

La línea futura principal, motivada cuantitativamente por tres hallazgos independientes, es sustituir ROS 2 en el bucle de entrenamiento por *bindings* directos sobre la dinámica del simulador, con llamadas explícitas de la forma estado–acción–paso temporal. Un simulador pausable eliminaría de raíz la degradación por teleports (el estado se asignaría directamente), la deriva por referencias persistentes (no habría tiempo real entre pasos) y el coste del *lockstep* vectorizado (los reinicios serían instantáneos), y permitiría escalar la vectorización a decenas de agentes aprovechando la inferencia por lotes. ROS 2 quedaría reservado para el despliegue de la política entrenada a la frecuencia correspondiente al paso temporal usado en entrenamiento.

Una segunda línea es corregir en su raíz la degradación del nodo de plataforma del fork ante reinicios de servicio repetidos, cuyo síntoma quedó cuantificado en este trabajo pero cuya causa interna no se investigó. Esta corrección beneficiaría a cualquier uso intensivo del servicio de reinicio, no solo al aprendizaje por refuerzo.

En el plano del aprendizaje, los resultados sugieren dos refinamientos: revisar la bonificación terminal para que la orientación final deficiente no resulte rentable (el error de yaw residual de 0,44 rad frente a los 0,01 rad del PID tiene origen en el diseño de la recompensa), y estudiar el escalado del curriculum hacia trayectorias con múltiples puntos de paso y referencias dinámicas. Sobre esa base, la extensión jerárquica planteada en los objetivos (un segundo controlador que transforme velocidades de-

Conclusiones y líneas futuras

seadas en tasas angulares) seguiría siendo la continuación natural hacia el control de bajo nivel.

Finalmente, la comparación con el controlador PID puede ampliarse en dos direcciones que delimitarían mejor el espacio donde el aprendizaje aporta ventaja: escenarios con perturbaciones (viento, ruido de estimación, retardos) y tareas sin ley de control evidente, donde el enfoque clásico requiere diseño manual específico y la política puede aprenderse con la misma infraestructura sin cambios.

Bibliografía

- AltexSoft (2021). *Reinforcement Learning Explained: Overview, Comparisons and Applications in Business*. URL: <https://www.altexsoft.com/blog/reinforcement-learning-explained-overview-comparisons-and-applications-in-business/> (visitado 28-06-2026).
- Anthropic (2026). *Claude Code*. URL: <https://claude.com/claude-code> (visitado 06-07-2026).
- Azar, Ahmad Taher, Anis Koubaa, Nada Ali Mohamed, Habiba A. Ibrahim, Zahra Fathy Ibrahim, Muhammad Kazim, Adel Ammar, Bilel Benjdira, Alaa M. Khamis, Ibrahim A. Hameed y Gabriella Casalino (2021). «Drone Deep Reinforcement Learning: A Review». En: *Electronics* 10.9, pág. 999. DOI: 10.3390/electronics10090999. URL: <https://doi.org/10.3390/electronics10090999>.
- Bengio, Yoshua, Jérôme Louradour, Ronan Collobert y Jason Weston (2009). «Curriculum Learning». En: *Proceedings of the 26th International Conference on Machine Learning (ICML)*, págs. 41-48.
- Chan, Jun Hoong, Kai Liu, Yu Chen, A. S. M. Sharifuzzaman Sagar y Yong-Guk Kim (2024). «Reinforcement Learning-based Drone Simulators: Survey, Practice, and Challenge». En: *Artificial Intelligence Review* 57.10, pág. 281. DOI: 10.1007/s10462-024-10933-w. URL: <https://doi.org/10.1007/s10462-024-10933-w>.
- Chollet, Gabriel (2021). *What is ROS*. URL: https://medium.com/@Gabriel_Chollet/what-is-ros-c38493fe3eca (visitado 28-06-2026).
- Eschmann, Jonas, Dario Albani y Giuseppe Loianno (2024). «Learning to Fly in Seconds». En: *IEEE Robotics and Automation Letters* 9.7, págs. 6336-6343. DOI: 10.1109/LRA.2024.3396025. URL: <https://doi.org/10.1109/LRA.2024.3396025>.
- Farama Foundation (2026). *Gymnasium Documentation: Env API*. URL: <https://gymnasium.farama.org/api/env/> (visitado 25-06-2026).
- Fernández-Cortizas, Miguel, Martín Molina, Pedro Arias-Perez, Rafael Perez-Segui, David Perez-Saura y Pascual Campoy (2023). «Aerostack2: A Software Framework for Developing Multi-robot Aerial Systems». En: *arXiv preprint arXiv:2303.18237*. DOI: 10.48550/arXiv.2303.18237. URL: <https://arxiv.org/abs/2303.18237>.
- Kaufmann, Elia, Leonard Bauersfeld y Davide Scaramuzza (2022). «A Benchmark Comparison of Learned Control Policies for Agile Quadrotor Flight». En: *2022 International Conference on Robotics and Automation (ICRA)*, págs. 10504-10510. DOI: 10.1109/ICRA46639.2022.9811564. URL: <https://doi.org/10.1109/ICRA46639.2022.9811564>.
- Kaufmann, Elia, Antonio Loquercio, Rene Ranftl, Matthias Müller, Vladlen Koltun y Davide Scaramuzza (2020). «Deep Drone Acrobatics». En: *Proceedings of Robotics: Science and Systems*. Corvallis, Oregon, USA. DOI: 10.15607/RSS.2020.XVI.040. URL: <https://www.roboticsproceedings.org/rss16/p040.html>.

- Krishnan, Srivatsan, Behzad Boroujerdian, William Fu, Aleksandra Faust y Vijay Janapa Reddi (2021). «Air Learning: a Deep Reinforcement Learning Gym for Autonomous Aerial Robot Visual Navigation». En: *Machine Learning* 110, págs. 2501-2540. DOI: 10.1007/s10994-021-06006-6. URL: <https://doi.org/10.1007/s10994-021-06006-6>.
- Molina, Martín (2025a). «Actor Critic Policy Optimization». Material docente de la asignatura Artificial Neural Networks and Deep Learning, Máster Universitario en Inteligencia Artificial, Universidad Politécnica de Madrid.
- (2025b). «Introduction to Reinforcement Learning». Material docente de la asignatura Artificial Neural Networks and Deep Learning, Máster Universitario en Inteligencia Artificial, Universidad Politécnica de Madrid.
- (2025c). «Proximal Policy Optimization». Material docente de la asignatura Artificial Neural Networks and Deep Learning, Máster Universitario en Inteligencia Artificial, Universidad Politécnica de Madrid.
- Objectways (2025). *Deep Learning Vs Reinforcement Learning: Key Differences*. URL: <https://objectways.com/blog/deep-learning-vs-reinforcement-learning-key-differences/> (visitado 28-06-2026).
- Open Robotics (2026). *ROS 2 Humble Documentation: Basic Concepts*. URL: <https://docs.ros.org/en/humble/Concepts.html> (visitado 25-06-2026).
- OpenAI (2026a). *ChatGPT*. URL: <https://chatgpt.com/> (visitado 02-07-2026).
- (2026b). *OpenAI Codex*. URL: <https://openai.com/codex/> (visitado 02-07-2026).
- Panerati, Jacopo, Hehui Zheng, Siqi Zhou, James Xu, Amanda Prorok y Angela P. Schoellig (2021). «Learning to Fly — a Gym Environment with PyBullet Physics for Reinforcement Learning of Multi-agent Quadcopter Control». En: *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, págs. 7512-7519. DOI: 10.1109/IROS51168.2021.9635857. URL: <https://doi.org/10.1109/IROS51168.2021.9635857>.
- Pi, Chen-Huan, Yi-Wei Dai, Kai-Chun Hu y Stone Cheng (2021). «General Purpose Low-Level Reinforcement Learning Control for Multi-Axis Rotor Aerial Vehicles». En: *Sensors* 21.13, pág. 4560. DOI: 10.3390/s21134560. URL: <https://doi.org/10.3390/s21134560>.
- ResearchGate (2024). *Proximal Policy Optimization (PPO) architecture*. URL: https://www.researchgate.net/figure/Proximal-Policy-Optimization-PPO-architecture_fig2_385920419 (visitado 28-06-2026).
- Sánchez-López, José Luis, Martín Molina, Hriday Bavle, Carlos Sampedro, Ramón Suárez Fernández, David Palacios, Antonio Díaz-Moreno y Pascual Campoy (2017). «A Multi-Layered Component-Based Approach for the Development of Aerial Robotic Systems: The Aerostack Framework». En: *Journal of Intelligent & Robotic Systems* 88.2-4, págs. 683-709. DOI: 10.1007/s10846-017-0551-4. URL: <https://doi.org/10.1007/s10846-017-0551-4>.
- Schulman, John, Filip Wolski, Prafulla Dhariwal, Alec Radford y Oleg Klimov (2017). «Proximal Policy Optimization Algorithms». En: *arXiv preprint arXiv:1707.06347*. DOI: 10.48550/arXiv.1707.06347. URL: <https://arxiv.org/abs/1707.06347>.
- Song, Yunlong, Selim Naji, Elia Kaufmann, Antonio Loquercio y Davide Scaramuzza (2021). «Flightmare: A Flexible Quadrotor Simulator». En: *Proceedings of the 5th Conference on Robot Learning*. Vol. 155. Proceedings of Machine Learning Research. PMLR, págs. 1147-1157. URL: <https://proceedings.mlr.press/v155/song21a.html>.

- Song, Yunlong, Angel Romero, Matthias Müller, Vladlen Koltun y Davide Scaramuzza (2023). «Reaching the Limit in Autonomous Racing: Optimal Control versus Reinforcement Learning». En: *Science Robotics* 8.82, eadg1462. DOI: 10.1126/scirobotics.adg1462. URL: <https://doi.org/10.1126/scirobotics.adg1462>.
- Stable-Baselines3 Contributors (2026). *Stable-Baselines3 Documentation: PPO*. URL: <https://stable-baselines3.readthedocs.io/en/master/modules/ppo.html> (visitado 25-06-2026).
- Sutton, Richard S. y Andrew G. Barto (2018). *Reinforcement Learning: An Introduction*. 2.^a ed. MIT Press. URL: <http://incompleteideas.net/book/the-book-2nd.html>.
- Towers, Mark, Ariel Kwiatkowski, Jordan Terry, John U. Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, Rodrigo Perez-Vicente, Andrea Pierré, Sander Schulhoff, Jun Jet Tai, Hannah Tan y Omar G. Younis (2024). «Gymnasium: A Standard Interface for Reinforcement Learning Environments». En: *arXiv preprint arXiv:2407.17032*. DOI: 10.48550/arXiv.2407.17032. URL: <https://arxiv.org/abs/2407.17032>.
- Yoo, Jaehyun, Dohyun Jang, H. Jin Kim y Karl H. Johansson (2021). «Hybrid Reinforcement Learning Control for a Micro Quadrotor Flight». En: *IEEE Control Systems Letters* 5.2, págs. 505-510. DOI: 10.1109/LCSYS.2020.3001663. URL: <https://doi.org/10.1109/LCSYS.2020.3001663>.

Apéndice A

Anexos

Los anexos recogen información complementaria para facilitar la reproducción y la inspección de los experimentos sin interrumpir la discusión principal. No se plantean como un registro exhaustivo de todas las ejecuciones, sino como una guía de lectura del repositorio: se resumen las configuraciones más relevantes, se indican los puntos de entrada de ejecución y se muestra un ejemplo de visualización de métricas en TensorBoard. A partir de esas referencias, cualquier ejecución puede consultarse directamente en los directorios de configuración, scripts y artefactos experimentales del repositorio.

A.1. Configuraciones representativas

Las configuraciones Exp007, Exp008 y Exp008a representan la evolución desde un entrenamiento largo bloqueado por reset hacia ejecuciones de curriculum con mayor instrumentación. Las configuraciones Exp009–Exp012 corresponden a la fase en la que el reinicio del simulador ya está certificado y el foco se desplaza hacia la validez de la recompensa, la randomización de episodios y la reducción de salidas de límites. Las configuraciones Exp013–Exp018 forman la campaña final: guards predictivos, curriculum de distancia en tres etapas y entrenamiento vectorizado con watchdog.

A.2. Comandos representativos

Config.	Objetivo	Duración	Rasgo principal
Exp007	Target desplazado estable.	20000 pasos.	Gate de reset tras eliminar fallback bloqueante.
Exp008	Target de 2 m.	20000 pasos.	Diagnóstico reforzado de reset y servicio del simulador.
Exp008a	Target de 0,5 m.	5000 pasos.	Curriculum inicial, yaw diagnóstico y <code>reset_max_vel</code> .
Exp009	Stack de reset certificado.	5000 pasos.	Primera ejecución PPO completa sobre el reinicio certificado.
Exp010	Reward mínima y randomización completa.	50000 pasos planificados.	Recompensa alineada con el tutor e hiperparámetros PPO revisados.
Exp011	Área segura y anti-saturación.	50000 pasos planificados.	Margen de muestreo y límite de distancia inicio-objetivo.
Exp012	Guards de límites y reanudación.	50000 pasos planificados.	Guard de acción, corredor vertical ampliado y supervisor con <i>resume</i> .
Exp013	Guards predictivos.	50000 pasos.	Disparo por posición + velocidad $\times 1$ s y empuje vertical con autoridad completa.
Exp014	Curriculum etapa 1.	50000 pasos.	Objetivos $a \leq 2,5$ m; primer aprendizaje sostenido.
Exp015	Curriculum etapa 2.	100000 pasos acumulados.	Objetivos $a \leq 3,5$ m con <i>warm start</i> de Exp014.
Exp016	Curriculum etapa 3.	150000 pasos acumulados.	Rango completo de 5 m; tarea original resuelta.
Exp017	Primer vectorizado (2 drones).	Detenido.	Colapso por deriva en <i>lockstep</i> ; origen del diagnóstico del watchdog.
Exp018	Vectorizado con watchdog.	200000 pasos acumulados.	2 drones en paralelo, reciclado preventivo y modelo final.

Cuadro A.1: Configuraciones representativas de la fase de estabilización.

A.2. Comandos representativos

La reproducción de una ejecución no depende de copiar literalmente un comando aislado, sino de combinar tres elementos del repositorio: el script que define el punto de entrada, el fichero YAML que fija la configuración del experimento y el directorio de salida donde quedan registrados los artefactos. Por este motivo, los comandos siguientes se presentan como patrones de uso y no como recetas cerradas. Antes de ejecutar pruebas con simulador conviene reiniciar AS2 para reducir el efecto de estados residuales del controlador o de la dinámica.

Listing A.1: Patrón general de entrenamiento PPO a partir de una configuración YAML.

```

1 python3 scripts/train_ppo.py \
2   --config configs/<configuracion_experimento>.yaml \
3   --num-envs <numero_entornos> \
4   --total-timesteps <pasos_entrenamiento>

```

La ejecución supervisada se utiliza en entrenamientos largos, especialmente cuando el simulador puede degradarse tras muchos reinicios. En este caso, el supervisor reinicia el stack de AS2, valida la disponibilidad del servicio de reinicio y permite reanudar desde un *checkpoint* o desde el modelo final de una etapa previa del curriculum.

Listing A.2: Patrón general de entrenamiento supervisado con reanudación.

```
1 RECYCLE_SECONDS=<segundos> \
2 DRONE_NAMESPACES=<drone0,drone1> \
3 INITIAL_RESUME=runs/ppo/<ejecucion_previa>/model_final.zip \
4 bash scripts/supervise_train.sh configs/<configuracion_experimento>.yaml
```

La evaluación formal utiliza el mismo punto de entrada para comparar políticas aprendidas y controladores de referencia sobre episodios equivalentes. La reproducibilidad de la comparación se apoya en mantener constantes la configuración, el número de episodios y la semilla inicial.

Listing A.3: Patrón general de evaluación por semillas.

```
1 python3 scripts/evaluate_policy.py \
2   --config configs/<configuracion_experimento>.yaml \
3   --model runs/ppo/<ejecucion>/model_final.zip \
4   --episodes <n_episodios> --seed <semilla>
5
6 python3 scripts/evaluate_policy.py \
7   --config configs/<configuracion_experimento>.yaml \
8   --controller pid --episodes <n_episodios> --seed <semilla>
```

La Tabla A.2 resume dónde consultar cada tipo de información cuando se quiera revisar una ejecución concreta, más allá de los ejemplos incluidos en este anexo.

Elemento	Ruta principal	Uso dentro del análisis
Configuraciones	configs/	Define recompensa, límites, curriculum, hiperparámetros y parámetros de ejecución.
Scripts de ejecución	scripts/	Contiene los puntos de entrada para entrenamiento, evaluación, supervisión y validaciones acotadas.
Modelos y checkpoints	runs/ppo/<ejecucion>/	Permite recuperar el modelo final, reanudar entrenamientos y auditar artefactos generados.
Monitores de episodio	runs/ppo/<ejecucion>/mon	Fuente principal para tasas de éxito, duración, recompensa, distancias y causas de terminación.
TensorBoard	runs/ppo/<ejecucion>/ten	Fuente complementaria para inspeccionar escalares de PPO y métricas agregadas durante el entrenamiento.

Cuadro A.2: Rutas de consulta de configuraciones, scripts y artefactos experimentales en el repositorio.

A.3. Capturas de TensorBoard

TensorBoard se utiliza en este trabajo como herramienta de inspección, no como fuente única de evidencia. Las curvas permiten observar la evolución de escalares registrados por Stable-Baselines3, como recompensa media, longitud media de episodio, tasa de éxito o métricas internas de PPO. Sin embargo, la interpretación de un experimento debe contrastarse siempre con los monitores de episodio y con las causas de terminación descritas en el Capítulo 6.

La Figura A.1 muestra un ejemplo de cómo se visualizan las métricas de *rollout* en TensorBoard para una ejecución concreta. No se incluyen capturas de todos los experimentos para evitar duplicar información ya disponible en el repositorio. Cuando se quiera revisar cualquier otra ejecución, basta con abrir el directorio de TensorBoard correspondiente al experimento deseado:

Listing A.4: Consulta de métricas TensorBoard para una ejecución concreta.

```
1 tensorboard --logdir runs/ppo/<ejecucion>/tensorboard
```

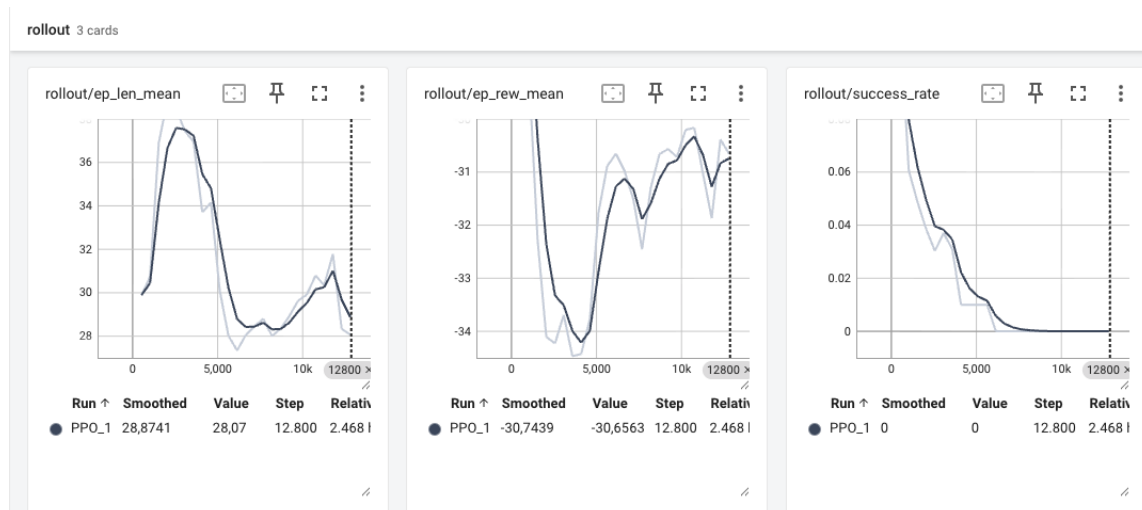


Figura A.1: Ejemplo de métricas de *rollout* de TensorBoard para Exp010. La captura ilustra el formato de consulta de los escalares registrados durante el entrenamiento; la interpretación del experimento se realiza junto con los monitores de episodio y la matriz de resultados.

A.4. Declaración de uso de herramientas de inteligencia artificial

Durante la realización del presente trabajo se han empleado herramientas de inteligencia artificial generativa como apoyo a tareas de programación, diagnóstico, organización de resultados y organización en la redacción. En particular, se ha utilizado OpenAI Codex en el entorno de desarrollo para asistencia sobre código, inspección del repositorio, generación de propuestas de modificación, revisión de errores y apoyo a la edición de la memoria [OpenAI, 2026b]. También se ha utilizado ChatGPT como asistente conversacional para estructurar explicaciones, revisar redacción y sintetizar criterios de presentación académica [OpenAI, 2026a]. Asimismo, se ha utilizado

Claude Code como agente de desarrollo en la fase final del trabajo, con apoyo a la parte experimental (supervisión de entrenamientos, análisis de logs y diagnóstico de fallos de integración) y a la implementación asistida de componentes con sus pruebas. [Anthropic, 2026].

El uso de estas herramientas no sustituye la autoría ni la responsabilidad académica del trabajo. Las decisiones de diseño, la interpretación de resultados, la selección de experimentos y la aceptación final de cambios se han revisado a partir del repositorio, de las trazas de ejecución, de los ficheros de configuración y de los logs experimentales. Las respuestas generadas por IA se han tratado como propuestas de trabajo que debían ser verificadas, corregidas o descartadas antes de incorporarse al documento o al código.

Herramienta	Tareas asistidas	Validación aplicada
OpenAI Codex	Inspección del repositorio, edición de LaTeX, revisión de errores de compilación y apoyo a cambios de código.	Revisión manual de los diffs, compilación de LaTeX y contraste con el repositorio.
ChatGPT	Estructuración de explicaciones, síntesis de resultados y apoyo a redacción académica.	Revisión del texto, eliminación de afirmaciones no respaldadas y contraste con evidencias experimentales.
Claude Code	Apoyo a la parte experimental, implementación asistida con pruebas automatizadas,	Revisión adversarial previa a cada integración, verificación de cifras contra los registros experimentales y revisión manual.

Cuadro A.3: Resumen del uso de herramientas de inteligencia artificial generativa.

Entre los prompts más relevantes se encuentran los dirigidos a analizar el estado real del repositorio y del registro experimental, a identificar diferencias entre resultados de entrenamiento y resultados de infraestructura. Por otra parte para revisar la compilación LaTeX y para organizar la incorporación de figuras y tablas.

No se han utilizado las herramientas de inteligencia artificial como fuente bibliográfica primaria sobre aprendizaje por refuerzo, ROS 2, Aerostack2 o Stable-Baselines3. Las afirmaciones técnicas del cuerpo de la memoria se apoyan en referencias bibliográficas, documentación técnica, código implementado y evidencia experimental. Cuando se ha incorporado texto redactado con apoyo de IA, este se ha revisado y adaptado para mantener el tono académico, la precisión terminológica y la coherencia con los resultados disponibles.